

Payment Decision and Execution Platform

Final consolidated architecture specification. Adds the Flow abstraction as the named unit of execution, single-difference variant construction, and the enforcement mechanisms that make experiment assignment and attribution correct by construction rather than by discipline.

Version	3.6 – consolidated
Status	Revised baseline
Supersedes	UPDP 2.0-2.2, spec 3.0-3.5
Scope	Auth path, money movement, control plane, learning layer

Contents

00	Constraints and what this design does not solve
01	Design principles
02	Reference architecture
03	Domain inventory
04	Resource model
05	Transaction context
06	Control plane
07	Execution model
08	Credential and instrument
09	Screening and financial crime
10	Platform security
11	Authentication
12	Routing and debit
13	Rail execution model
14	Attempt strategy
15	Recovery and compensation
16	Learning layer
17	Decision platform
18	Experimentation
19	Money movement
20	Cross-border, FX and multi-currency
21	State machines
22	Journal and determinism
23	Error model
24	Observability and disclosure
25	API surface
26	Latency and data architecture
27	Resilience and deployment
28	Migration and legacy coexistence
29	Commerce device platform
30	Vertical extensions
31	Unresolved questions
A	Change log and review disposition

Constraints and what this design does not solve

Six constraints shape every decision below. They are stated first because most objections to this architecture are really objections to one of them, and it is cheaper to argue about the constraint than about the design that follows from it.

The six binding constraints

We are a bank, not a fintech. Every decisioning module that uses a model is subject to model risk governance. Every transaction passes sanctions screening. Every data flow is subject to residency rules. These are not features to be added later; they sit on the authorization path and they consume latency budget. A design that reaches a latency number by omitting them has not reached the number.

We are late. Stripe and Adyen have a decade of production learning curve encoded in routing tables, retry heuristics and decline taxonomies. We cannot out-iterate that from a standing start with hand-tuned rules. The only structural answer is to make the platform learn faster per unit of volume than they did, which is why the learning layer (§16) is load-bearing rather than an enhancement.

Our volume advantage is real but concentrated. We see enormous card volume, but disproportionately from large enterprise merchants in a narrow set of verticals. Learned strategy will be strong where we are dense and weak where we are thin. The design must fail safe into policy and heuristic where evidence is insufficient, and must expose which regime a given transaction was decided under.

Migration is not greenfield. The existing engine-centric stack processes live volume and will continue to for years. Every abstraction here must be expressible as a facade over the legacy path so that the resource model can go live before the execution engine does.

Enterprise merchants demand determinism. The merchants we are best positioned to win are precisely the ones least tolerant of unexplained behavioural variance. Autonomous optimization and enterprise sales pull in opposite directions. §06 and §16 resolve this by making autonomy an entitlement that a merchant can decline, not a platform-wide setting.

The unit of competition is not the gateway. Acceptance rate, cost of acceptance, and time-to-diagnose are the three things a sophisticated merchant actually buys. Anything in this document that does not serve one of those three is scaffolding.

Simplicity is a requirement, not a preference. The commercial case rests on acceptance rate, cost of acceptance and time-to-diagnose — but none of those matter if a developer cannot integrate in an afternoon. Depth of model and simplicity of surface are in permanent tension, and where they conflict the surface wins: the resource model may be five layers deep, but the common path must be one call (§04, §25).

Deliberate non-goals

- **Exactly-once processing.** Not attempted. At-least-once delivery, idempotent commands and explicit deduplication (§22) are safer and honest about the distributed reality.
- **A universal ledger for the enterprise.** This platform owns the payment ledger and its reconciliation to settlement. It does not attempt to be the bank's general ledger.
- **Real-time model training.** Decisions are served from versioned artefacts. Training is offline, governed and slow on purpose (§16).
- **Full processor abstraction on day one.** The adapter layer (§12) normalises what can be normalised. Dual-message versus single-message semantics leak, and the design admits the leak rather than pretending otherwise.

Known weak points in this document

Security architecture is named as a domain and not specified — key management, cryptographic boundary decomposition and service-to-service authorization are assumed to be inherited from bank standards, which is not the same as being designed for this platform. It is the largest omission here. The dispute lifecycle is sketched. Settlement and funding (§19) remain shallower than the authorization path. Non-card rails have an owner but not a design. Card-present terminal integration is assumed. Section 31 lists these and ten others rather than burying them.

Design principles

A payment is an immutable business intent. An execution plan determines which versioned capabilities run, in what order, under which constraints — with every decision logged with enough context to be replayed, attributed and second-guessed.

The four separations

The single most important structural claim in this architecture is that four things which are conflated in a conventional gateway must be separated, because they have different owners, different rates of change, and different failure semantics.

Concern	Question it answers	Changes when	Owner
Decisioning	What should happen to this transaction?	Policy, model or strategy changes	Product / Risk / Compliance
Orchestration	In what order do capabilities run, and what happens when one fails?	Plan version changes	Platform engineering
Capability execution	How is one decision computed or one action performed?	Module version changes	Capability team
State management	What is true about this payment now, and what was true before?	Almost never	Platform engineering

Where a conventional gateway lets these bleed together, the result is the monolith we are trying to leave: a routing change requires a release of the payment engine, and a payment engine release re-tests routing.

Module contract

Every capability module satisfies the same contract. A module that cannot satisfy it is either badly scoped or is actually an orchestration concern in disguise.

```
module(input: TransactionContext + explicit dependencies)
-> Decision | Effect | Failure

Required properties
stateless      no mutation of payment state
versioned      semver, deployable independently
deterministic  same input + same dependency versions
               -> same output (models included:
                   artefact version is an input)
idempotent     for modules producing effects
timeout-controlled hard deadline, declared not discovered
observable     emits a ModuleExecutionRecord always,
               including on timeout and failure
fallback-declared states its own degraded behaviour;
               the orchestrator does not invent one
```

The prohibited shape, stated explicitly because it is what every module drifts toward under delivery pressure:

```
RoutingModule
-> reads database
-> mutates payment
-> calls processor
-> updates authentication state
-> publishes event           # this is a monolith wearing a module costume
```

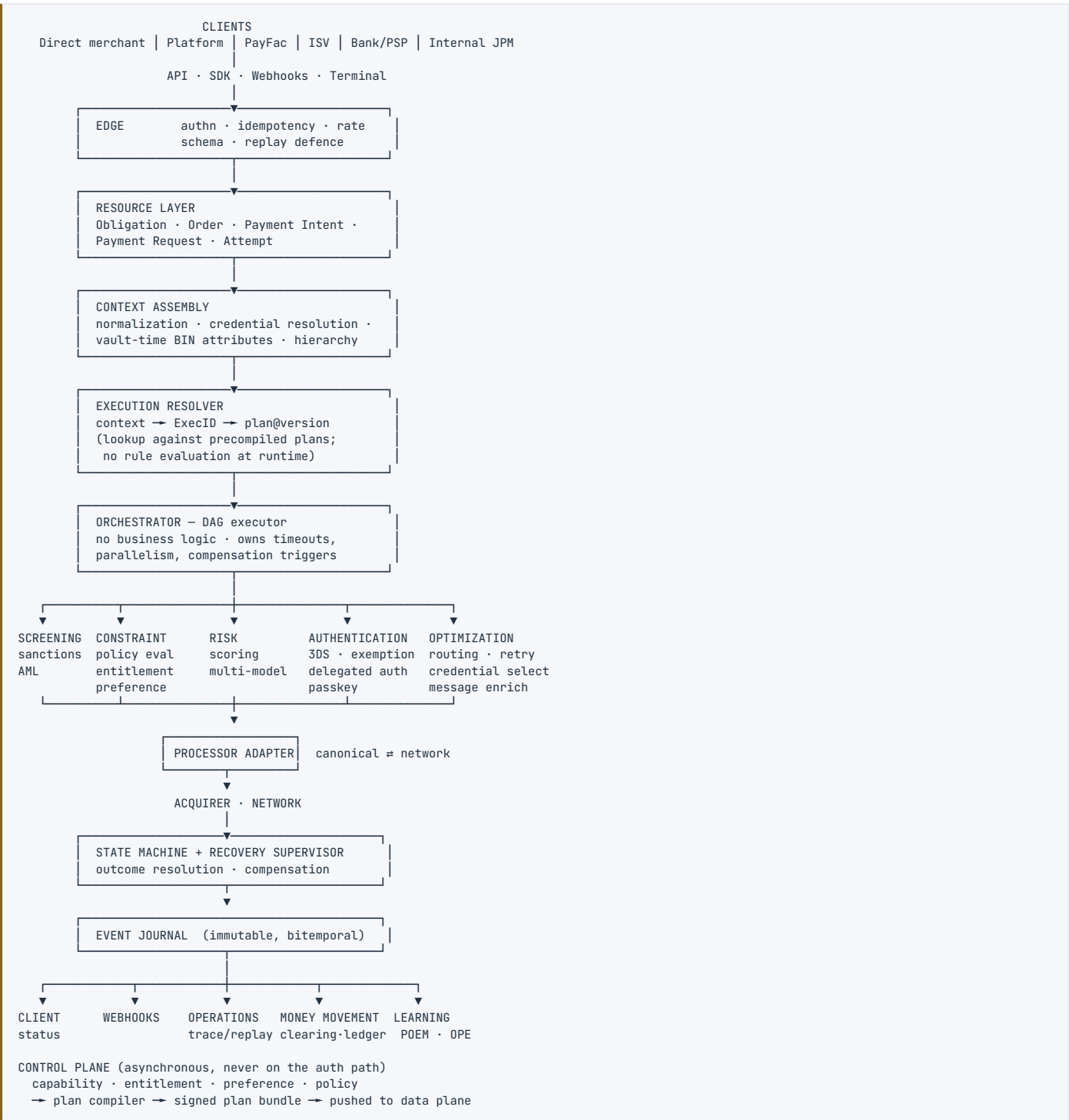
Nine principles

1. **Intent is immutable; execution is versioned.** A payment's business meaning never changes. How we tried to fulfil it is a versioned, replayable artefact.
2. **Every decision carries provenance.** Decision, decider, version, inputs, reason code, and the regime it was decided under. A decision without provenance is unusable for learning, dispute or audit.
3. **Constraints are hierarchical and non-negotiable downward.** Regulation cannot be overridden by optimization (§06).
4. **Learning is bounded by entitlement.** A merchant can decline autonomy; the platform must still work well without it.
5. **Precompute over lookup.** Nothing on the authorization path performs a remote lookup for information that could have been resolved at vault time or config-push time (§08, §26).
6. **Recovery is a domain, not an afterthought.** Someone owns the unknown-outcome timeout. It is not the module (§15).

7. **Internal state vocabulary is large; external is small.** We do not export our complexity to merchants (§21).
8. **Disclosure is a function of role and delegation, not a table of booleans.** A PayFac's view of a submerchant is a policy question (§24).
9. **Degrade explicitly.** Every dependency failure resolves to a declared, merchant-visible-in-principle behaviour, not to an incident.

Reference architecture

Two planes, cleanly separated. The **control plane** resolves what a client is capable of, entitled to, prefers, and is constrained by — and compiles that into execution plans ahead of time. The **data plane** executes plans at transaction speed and never asks the control plane a synchronous question.



Change from v2.2

Screening is now a named first-class stage on the authorization path rather than an assumed platform service, and the Recovery Supervisor is a named component rather than behaviour distributed across the orchestrator and state machine. Both changes came out of the v3 gap review.

Domain inventory

Thirty-three bounded contexts. Each is a product with an owner, a roadmap, an SLA and — for the ten marked as externally monetisable — a commercial model. The count matters: fewer and we recreate the monolith; more and the coordination cost exceeds the modularity benefit.

Domain	Responsibility	On auth path	Sellable
Resource	Obligation, Order, Intent, Request, Attempt lifecycle	Yes	—
Execution Resolver	Context → ExecID → plan binding	Yes	—
Orchestrator	DAG execution, timeout, parallelism	Yes	—
Plan Compiler	Compiles control-plane state into plans	No	—
Capability Registry	What the platform can do, where	No	—
Entitlement	What this client may use	Cached	—
Preference	What this client wants by default	Cached	—
Policy	Constraints and their precedence	Cached	Yes
Screening	Sanctions, AML, watchlist	Yes	—
Payment Methods and Rails	Non-card rails: ACH, RTP, FedNow, wire, open banking, wallets, transfers	Yes	—
Credential Vault	PAN and token lifecycle, references	Yes	Yes
Payment Instrument Intelligence	BIN, PIH, eligibility resolution	Precomputed	Yes
Authentication	3DS, exemption, delegated, passkey	Yes	Yes
Risk	Fraud and abuse scoring	Yes	Yes
Routing	Acquirer, network, processor selection	Yes	Yes
Message Intelligence	Authorization message enrichment	Yes	Yes
Attempt Strategy	What to try next after a failure	Yes	Yes
Recovery	Compensation, reversal, orphan resolution	Async	—
Processing	Authorize, capture, refund, void	Yes	—
Processor Adapter	Canonical ⇄ network message translation	Yes	—
Cross-Border & FX	Rate locks, currency conversion, local acquiring, multi-currency exposure	Rate lock only	Yes
Clearing & Settlement	Presentment, settlement lifecycle	No	—
Funding	Merchant payout, reserves, FBO structures	No	—
Ledger	Double-entry payment accounting	No	—
Pricing	Interchange, scheme fees, merchant pricing	No	Yes
Disputes	Chargeback and representment lifecycle	No	—
Learning	POEM, experiment assignment, off-policy evaluation	Assignment only	—
Decision Platform	Strategy authoring, backtest, promotion, attribution, registry	No	Yes
Model Governance	Validation, monitoring, SR 11-7 evidence	No	—
Developer Platform	API surface, SDKs, progressive disclosure, webhooks, sandbox	No	—
Platform Security	Cryptographic zones, key hierarchy, workload identity, replay boundary (\$10)	Yes	—
Commerce Devices	Terminal estate, key injection, firmware, offline operation (\$29)	Yes	—
Network Intelligence	Issuer, BIN, acquirer and outcome graph — the data asset behind routing and authentication	Precomputed	Yes

This specification is card-shaped

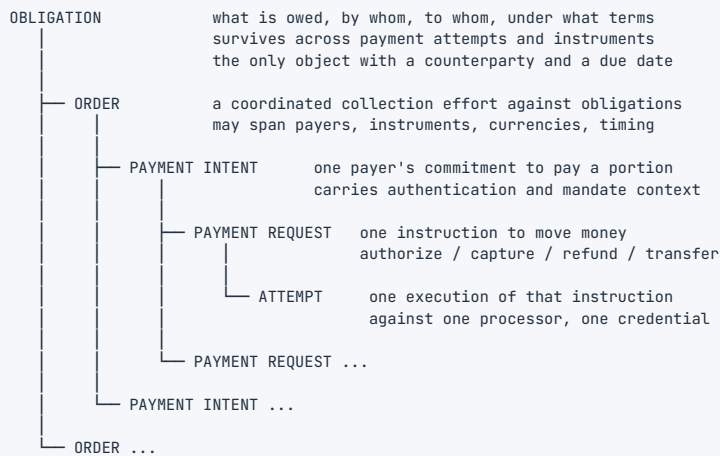
The execution model generalises to any rail in principle, and does so cleanly for anything with an authorize-then-capture shape. It generalises poorly to rails whose semantics differ at the core: push payments have no authorization to reverse, real-time rails are irreversible on receipt, and open banking puts authentication upstream of the payment rather than inside it. Recovery (§15) and Attempt Strategy (§14) are the two places where the card assumption is load-bearing. The Payment Methods and Rails domain owns closing that gap; §31 item 9 tracks it. Claiming full rail coverage today would be false.

Why Attempt Strategy replaced Retry

v2.2 had a Retry domain and an Authentication domain that could not see each other. That made 3DS-as-retry — stepping up to authentication in response to a soft decline rather than giving up — structurally impossible to express. In v3 there is one domain that owns "what do we try next," and step-up authentication is one of its available actions alongside credential substitution and processor substitution. See §14.

Resource model

Five nested resources. The layering exists because merchants ask five different questions — *what is owed, what is being collected, what did the customer agree to, what did we send, what happened* — and a design that collapses them forces every complex commercial structure into metadata.



Progressive disclosure – the depth must be optional

Five layers is the right *model* and the wrong *default surface*. A developer taking a card payment on a checkout page must never create an obligation or an order. The common path is one call, and the platform provisions the layers above it implicitly. Depth is opt-in, reached only by clients whose commercial structure needs it. §25 specifies the API consequence.

Standard checkout – what the developer writes

```
POST /v1/payment_intents
{ "amount": 12500, "currency": "USD",
  "payment_method": "pm...", "capture_method": "automatic" }
```

What the platform creates

Obligation (implicit)	customer → merchant, basis=sale, discharged by this intent alone
Order (implicit)	single-payer, single-instrument
PaymentIntent	the object the developer holds
PaymentRequest	AUTHORIZE
Attempt	one per execution

Implicit objects are real objects, not hidden state. They are addressable, retrievable and appear in reporting exactly as explicit ones do. This is what keeps the reporting and settlement model uniform across simple and complex clients – the alternative, two code paths, is how platforms end up with reconciliation that works for one customer segment and not the other.

Explicit construction is required only when the developer needs something the implicit shape cannot express: multiple payers, splits that outlive the collection, schedules, or obligations between parties other than customer and merchant.

Why Obligation exists

Obligation is the object that competitors do not have, and it is the reason our resource model can express structures theirs cannot without bespoke integration work.

Structure	Without Obligation	With Obligation
Franchise royalty	Franchisor reconciles off-platform from settlement reports	Obligation from franchisee to franchisor, satisfied from collected funds pre-payout
Marketplace split	Transfer objects chained after the fact	One collection, several obligations discharged in one settlement instruction
Subscription dunning	Failed invoice, new payment, merchant-side linkage	Obligation persists; new intents attach until discharged or written off
Instalments	N unrelated payments plus merchant bookkeeping	One obligation, a schedule, N intents, an explicit outstanding balance
Deposit then balance	Two payments the platform cannot relate	One obligation, partial discharge, known remainder
B2B invoice with partial payment	Not expressible	Native

```
Obligation
├── obligation_id
├── obligor      {customer | merchant | submerchant | platform}
├── obligee      {merchant | platform | franchisor | PSP}
├── original_amount / outstanding_amount / currency
├── basis        {sale, subscription, royalty, fee, settlement,
                 adjustment, penalty, refund_liability}
├── schedule     due dates, instalment terms
├── discharge_policy what satisfies it, in what order
├── state        OPEN · PARTIALLY_DISCHARGED · DISCHARGED
                 · DISPUTED · WRITTEN_OFF · CANCELLED
├── linked_orders[]
└── ledger_account_ref
```

Payment Intent versus Payment Request

The distinction the reviewed design misses. An intent is the customer's commitment and carries the authentication and mandate context. A request is one instruction we send. One intent can produce many requests over its life — an authorization, an incremental authorization, a partial capture, a refund — and each of those is separately addressable, separately retryable, and separately reportable.

```
PaymentIntent
├── intent_id · order_id · obligation_refs[]
├── payer      customer_id | account
├── amount_authorized_ceiling · currency
├── instrument_ref (vault reference, never a credential)
├── mandate      CIT/MIT · agreement · network transaction id
├── authentication_context (see §11 – bound to the intent, not
                           the request, so it survives retries)
├── capture_method · statement_descriptor
├── industry_context_ref (see §30)
└── state

PaymentRequest
├── request_id · intent_id
├── operation   AUTHORIZE · INCREMENTAL · CAPTURE · PARTIAL_CAPTURE
               · VOID · REFUND · REVERSAL · TRANSFER_IN · TRANSFER_OUT
├── amount
├── idempotency_key
├── exec_id · plan_version (bound at resolution)
├── attempts[]
└── state
```

Attempt

The attempt is where diagnosis happens, and it is the single richest object in the system. Every decision that shaped one network message is recorded here, with the version of whatever made it.

```
Attempt
├── attempt_id · request_id · sequence
├── exec_id · plan_version
├── decisions{
│   ├── screening · policy · risk · authentication ·
│   │   credential_selection · message_enrichment · routing
│   └── each with {decision, version,
│                 reason_code, regime, latency_ms}
├── strategy_origin why this attempt exists: INITIAL |
                   STEP_UP_AUTH | CREDENTIAL_SUB |
                   PROCESSOR_SUB | NETWORK_SUB | SCHEDULED_RETRY
├── processor · acquirer · network · merchant_account
└── request_message_ref · response_message_ref (restricted, §24)
```

```
| canonical_result · processor_code · network_code
| timings{ dispatched, first_byte, completed }
| outcome APPROVED | DECLINED | ERROR | TIMEOUT | UNKNOWN
| recovery_ref      (set when outcome is UNKNOWN, §15)
```

Migration note

The resource model can ship ahead of the execution engine. Obligation, Order, Intent and Request are expressible as a facade over the legacy stack, with `exec_id` and the decision block populated as constants. That lets the API, the reporting model and the merchant-facing contract stabilise a year before the orchestrator carries production volume.

Transaction context

The context is assembled once, immutable thereafter, and is the sole input to plan resolution and every module. If a module needs something not in the context, either the context is wrong or the module is reaching outside its contract.

TransactionContext	
HIERARCHY	psp · legal_entity · account · platform · payfac · isv · merchant · submerchant · location · terminal · customer
COMMERCIAL	obligation_basis · order_shape · industry · MCC · pricing_plan · contract_terms_ref
GEOGRAPHY	merchant_country · acquiring_country · issuer_country · billing · shipping · IP · data_residency_zone
MONETARY	amount · presentment_currency · settlement_currency · FX_treatment · amount_band
INSTRUMENT	method · network · funding_type · product_id · commercial · prepaid · regulated_debit · HSA/FSA · token_type · token_requestor · wallet · credential_age · updater_status ← resolved from vault-time PIH enrichment, §08
CHANNEL	CP/CNP · ecom · MOTO · in-app · terminal · QR · unattended · entry_mode · PIN_capability
MANDATE	CIT/MIT · recurring · instalment · COF · network_transaction_id · agreement_ref
CONSTRAINT	resolved constraint set from control plane (§06) including SCA applicability and Reg II posture
OPERATION	requested operation and its parameters
EXPERIMENT	POEM assignment vector (§16) — assigned at context assembly so it is available to every decision point and logged with propensity

Ordering correction

Instrument attributes are resolved **before** constraint evaluation, not after. The reviewed design places policy at step 3 and BIN at step 4, which makes it impossible to evaluate a debit-scoped, regulated-debit-scoped or product-scoped constraint — the most common kind we have. In v3 the instrument attributes arrive with the credential from vault-time enrichment, so the ordering problem disappears entirely rather than being solved with a runtime lookup.

Effective configuration snapshot

The client hierarchy resolves eleven levels, and resolving that graph per transaction would put a graph traversal on the authorization path. It is resolved ahead of time into one immutable snapshot, which is what the context actually carries.

EffectiveConfigSnapshot	
snapshot_id · version · valid_from	
capability · entitlement · preference · policy	(resolved)
routing · pricing · risk posture	
active strategies and their populations	
experiment layer assignments available	
signature	

Recomputed on any contributing change, pushed like a plan bundle (§07), consumed as a single in-memory read. The hierarchy is a control-plane concern; the data plane sees only its result.

Control plane

The most significant structural difference between this design and a conventional one. A conventional gateway has a rules engine. We have four planes that answer four different questions, resolved in a fixed order, each with a different owner and change process.

Plane	Question	Owner	Change velocity	Override
Capability	Can the platform do this here, at all?	Platform engineering	Release cadence	Never
Entitlement	Is this client permitted and contracted for it?	Commercial / risk	Contract events	Approval workflow
Preference	What does this client want by default?	Client, self-serve	Continuous	Client, freely
Policy	What constrains this regardless of preference?	Compliance / risk / scheme	Regulatory events	Governed exception only

Resolution order is `capability → entitlement → policy → preference`. A preference that exceeds an entitlement is not an error; it is inert, and it is reported as inert so the merchant can see that their setting is having no effect. This single behaviour eliminates a large class of support tickets that plague competitor platforms, where a merchant sets a flag and cannot tell whether it did anything.

Authority ladder

Eight levels. Higher levels bind lower ones unless authority is explicitly delegated downward with a recorded delegation grant.

1

REGULATOR

SCA · Reg II · sanctions · residency · limits

2

NETWORK/SCHEME

scheme rules, mandates, prohibited practice

3

PSP INSTITUTIONAL

JPM risk appetite, prohibited MCC, exposure limits

4

PLATFORM

platform operator rules over its estate

5

PAYFAC

PayFac rules over its submerchants

6

MERCHANT

merchant configuration

7

SUBMERCHANT

submerchant configuration

8

LEARNED STRATEGY

optimization operating inside 1-7

Delegation grant

{

"grant_id": "...",

"from_level": "PAYFAC",

"to_level": "SUBMERCHANT",

"scope": ["authentication.challenge_preference"],

"bounds": { "may_not": ["disable_3ds_in_psd2_market"] },

"expires_at": "..."

}

Non-negotiable

Learned strategy sits at level 8 — below every human and regulatory authority. Optimization chooses among options that levels 1–7 have already declared acceptable. It never expands the option set. This is what makes the learning layer defensible to model risk, to compliance, and to an enterprise merchant's procurement team.

Bitemporal records

Every control-plane record carries two time dimensions. Without both, replay is a lie: you can reconstruct what we would decide today, but not what we decided then, and you cannot distinguish a policy that changed from a policy we recorded late.

{

"record_id": "pol_8813",

"record_version": 8,

"valid_from": "2026-01-01T00:00:00Z", // when the rule applies

"valid_to": "2026-06-30T23:59:59Z", // in the real world

"recorded_at": "2025-11-14T09:22:10Z", // when we learned it

"superseded_at": null, // when our record changed

"authority_level": "REGULATOR",

"jurisdiction": "EEA",

"scope": { "channel": "ecommerce", "amount_gt": 3000 },

"constraint": { "type": "REQUIRE_SCA", "exemptions_permitted": ["TRA", "LOW_VALUE"] },

"provenance": { "source": "PSD2 RTS Art.16", "approved_by": "..."}

}

The three questions this makes answerable, none of which single-versioned policy can answer:

- *What rule was in force when this transaction ran?* — valid time at transaction timestamp.
- *What did we believe was in force when we ran it?* — transaction time at transaction timestamp. If these differ, we misapplied a rule and can quantify the exposure.
- *Which transactions were decided under a rule we later corrected?* — the retroactive-correction query, which is the one that matters in a regulatory finding.

Policy algebra

Constraints resolve to a small set of verbs with defined composition, so that the action space at any decision point is a computed set rather than the outcome of a rule engine's traversal order.

Verbs ALLOW · DENY · REQUIRE · PROHIBIT · PREFER · BOUND · DELEGATE

Composition, by authority level descending (§06 ladder)
 DENY and PROHIBIT are absorbing – a lower level cannot restore
 REQUIRE narrows to the required subset
 BOUND intersects numeric ranges
 PREFER orders within whatever remains; it never adds
 DELEGATE transfers authority for a named scope, with bounds

Resulting action space at a decision point

```
ActionSpace = Capability
               n Entitlement
               n AllowedActions      (after DENY / PROHIBIT)
               n RequiredActions     (after REQUIRE)
               n Bounds              (after BOUND)
               ordered by PreferredActions
```

Strategies (§17) select within ActionSpace. They cannot expand it, and because it is computed rather than interpreted, that property is checkable at compile time rather than enforced at runtime.

Emergency propagation

Compiled plans and pushed constraint sets are the normal path, and they are eventually consistent by design. That is acceptable for a merchant preference change and unacceptable for a sanctions addition or an emergency scheme mandate. A second channel exists for exactly that case, and its scope is deliberately narrow.

NORMAL PATH	compile → sign → push → adopt propagation SL0: 60 s p99 global scope: everything
EMERGENCY PATH	signed constraint delta → direct to edge propagation SL0: 5 s p99 global scope: PROHIBIT and REQUIRE constraints only – never preferences, never plan changes, never routing or model artefacts authority: REGULATOR, NETWORK or PSP levels only dual control: two approvers, both logged always additive: an emergency delta may restrict, never permit

Fail-closed property

A node that cannot confirm it has applied an emergency delta within its deadline stops accepting traffic for the affected scope rather than processing under a constraint set it knows to be superseded.

Last-known-good expiry

Every pushed artefact carries a TTL. On expiry a node degrades to the most restrictive constraint set it holds, and reports degraded. There is no configuration under which a node processes indefinitely on an artefact it can no longer refresh.

Constraint resolution output

Resolution produces a constraint set attached to the context, not a stream of runtime rule evaluations. The plan compiler does the expensive work offline.

```
ResolvedConstraintSet
├─ required[]    REQUIRE_SCA, REQUIRE_PIN, REQUIRE_NETWORK_TOKEN
├─ prohibited[] NO_STORED_CREDENTIAL, NO_CROSS_BORDER_ROUTE
├─ bounded[]     MAX_RETRY_ATTEMPTS=3, MAX_AMOUNT=..., VELOCITY=...
├─ preferred[]   PREFER_LEAST_COST_ROUTE, PREFER_NETWORK_TOKEN
├─ autonomy     { routing: AUTO, retry: AUTO, auth: ADVISORY }
```

└─ provenance[] one entry per contributing record, with
authority level and bitemporal coordinates

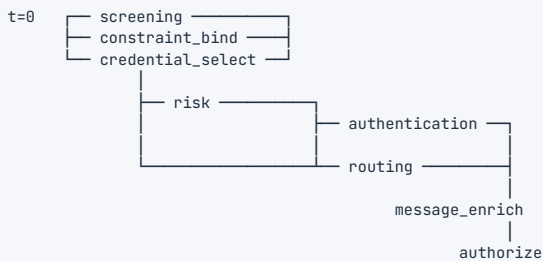
Execution model

The execution identifier is the platform's central abstraction. It is an opaque handle to a compiled plan, not an encoded string — encoding semantics into the identifier is a trap that makes the taxonomy unchangeable.

```
ExecutionPlan
{
  "exec_id": "exe_01JQ7F3M2K",
  "plan_version": 42,
  "profile": "ECOM_CARD_AUTH_US",
  "compiled_at": "...",
  "compiled_from": {
    "capability_set": "v118",
    "entitlement_snapshot": "v9042",
    "policy_set": "v311",
    "preference_snapshot": "v88214"
  },
  "graph": [
    { "node": "screening", "version": "4.1", "deadline_ms": 12,
      "depends_on": [], "on_failure": "HARD_FAIL" },
    { "node": "constraint_bind", "version": "12.1", "deadline_ms": 2,
      "depends_on": [], "on_failure": "HARD_FAIL" },
    { "node": "risk", "version": "19.4", "deadline_ms": 18,
      "depends_on": ["constraint_bind"], "on_failure": "FALLBACK" },
    { "node": "authentication", "version": "31.0", "deadline_ms": 6,
      "depends_on": ["risk", "constraint_bind"], "on_failure": "FALLBACK" },
    { "node": "credential_select", "version": "8.3", "deadline_ms": 3,
      "depends_on": ["constraint_bind"], "on_failure": "FALLBACK" },
    { "node": "routing", "version": "23.1", "deadline_ms": 4,
      "depends_on": ["risk", "credential_select"], "on_failure": "FALLBACK" },
    { "node": "message_enrich", "version": "11.7", "deadline_ms": 3,
      "depends_on": ["routing", "authentication"], "on_failure": "DEGRADE" },
    { "node": "authorize", "version": "18.2", "deadline_ms": 8000,
      "depends_on": ["message_enrich"], "on_failure": "RECOVER" }
  ],
  "compensation": { ... see §15 ... },
  "signature": "..."
}
```

Parallelism

Screening, constraint binding and credential selection have no mutual dependencies and run concurrently. Risk gates authentication because the authentication decision consumes the score. Routing consumes both risk and the selected credential.



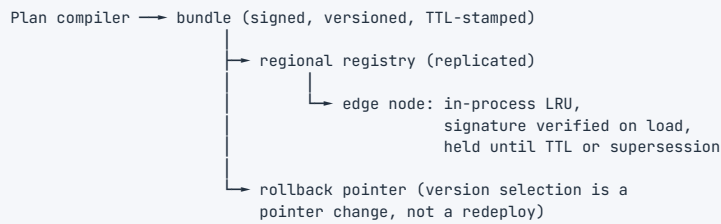
Plan versioning and traffic

Plans are versioned artefacts. A new plan version can be exposed to a slice of traffic defined by any context dimension — not only a percentage. This is the macro-experiment mechanism; §16 covers the micro mechanism.

```
plan ECOM_CARD_AUTH_US
v41 → 87% stable
v42 → 12% general ramp
v43 → 1% canary, excluding: strategic_accounts,
                             PSD2_markets,
                             amount > $10,000
v44 → 0% internal dogfood only (JPM-as-merchant traffic)
```

Plan distribution

Plans are compiled centrally and distributed as signed bundles. The data plane never resolves a plan by asking the control plane a question, so control-plane unavailability degrades freshness rather than availability.



Cache behaviour	
positive cache	resolved plan held for TTL
negative cache	"no plan for this context shape" is itself cached, so an unusual context cannot generate repeated compile requests
request coalescing	single in-flight fetch per key per node – a cold key under load produces one fetch, not N
pre-warm	new plan versions are distributed before their exposure predicate is switched on, so ramping traffic never triggers a cold fetch
Failure behaviour	
compiler down	existing bundles keep serving until TTL
registry down	edge serves from LRU, reports degraded
bundle expired	node degrades to most restrictive held set (§06)
signature invalid	bundle rejected, node keeps prior version, alerts

Module execution record

Emitted for every node on every attempt, including timeouts. This is simultaneously the diagnostic object, the audit record and the training row — one artefact, three consumers, which is why its schema is governed.

```
{
  "attempt_id": "att...", "node": "routing", "version": "23.1",
  "status": "completed",           // completed | fallback | timeout | error
  "started_at": "...", "duration_ms": 2.8,
  "input_digest": "sha256:...",   // full input retained per retention policy
  "decision": {
    "processor": "processor_a",
    "reason_code": "AUTH_RATE_OPTIMIZATION",
    "regime": "LEARNED",           // POLICY | HEURISTIC | LEARNED | FALLBACK
    "considered": ["processor_a", "processor_b", "processor_c"],
    "propensity": 0.72             // §16 – required when regime=LEARNED
  },
  "dependencies": {
    "routing_table": "v184", "bin_snapshot": "v52",
    "model": "route_auth_v19", "policy_set": "v311"
  },
  "experiment": { "poem_cell": "routing_arm_b", "assignment_id": "..." }
}
```

Adopted from the review

Carrying **dependencies** inside the record itself, rather than reconstructing them from the plan, is a clean idea taken from the reviewed document. It makes each record independently interpretable years later, which matters for dispute reconstruction where the plan artefact may have been archived separately.

Credential and instrument

The vault exposes references. Credentials never enter application logs, never appear in module inputs, and never cross a service boundary except to an adapter inside the PCI boundary.

```
Credential
├── credential_id           the only thing modules ever see
├── scope { merchant | platform | psp | customer }
├── type  FPAN · DPAN · MPAN · acquirer_token · psp_token
├── network · token_requestor_id · token_reference
├── lifecycle_state
├── last4 · expiry · credential_age
├── updater_status · network_token_status
├── ENRICHED ATTRIBUTES ← resolved at vault time from PIH
│   ├── issuer · issuer_country · product_id
│   ├── funding_type · commercial · prepaid
│   ├── regulated_debit · debit_networks[]
│   ├── pinless_eligibility { eligible, networks[], basis, as_of }
│   ├── HSA/FSA indicator
│   └── enrichment_version
```

Vault-time enrichment

The single highest-leverage latency decision in the architecture. BIN attributes are resolved when a credential is vaulted or when the BIN snapshot changes — never on the authorization path.

```
Vault time (asynchronous, unbounded latency budget)
  credential stored
    ├── PIH lookup: Visa ARDEF/VBAS/PRBIN · MC SBART/MPE · Discover DCI
    ├── US debit overlay resolution
    ├── PINless eligibility resolver (versioned pure function)
    └── attributes written alongside the credential reference

BIN snapshot v52 → v53
  └── re-enrichment sweep over affected credential ranges,
      versioned, with before/after diff retained

Authorization time
  credential_id → attributes already present  (0 ms, in-memory)
```

The eligibility resolver is specified as a versioned pure function over the BIN snapshot and the merchant's debit configuration. That property is what makes PINless routing decisions reproducible eighteen months later during a network audit, and it is why the resolver is not permitted to consult live state.

Freshness — the honest limit of precomputation

Precomputed attributes are stale attributes. Issuers sell portfolios, reissue ranges and change debit participation continuously, and a snapshot taken last week can be wrong about a card presented today. The design does not pretend otherwise; it makes staleness measurable and makes the decision layer account for it.

```
Every enriched attribute carries
  value · source · as_of · enrichment_version · freshness_class

Freshness classes — different attributes decay at different rates,
so a single global refresh window would be both too slow for one
and wasteful for another.

VOLATILE      debit network participation, PINless eligibility,
               regulated status
               refresh: on every network file delta affecting the
               range; verify on first use after 24h
SEMI_STABLE   product id, commercial flag, prepaid flag
               refresh: weekly sweep, plus on delta
STABLE        issuer country, network
               refresh: on delta only

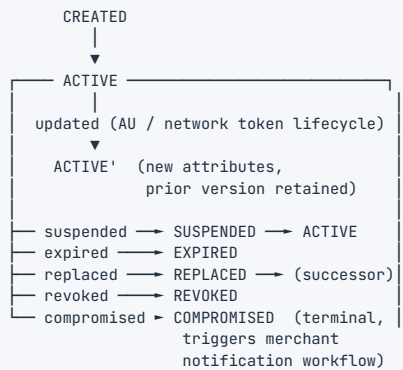
Invalidation channels
1 scheduled sweep          baseline coverage
2 network file delta      event-driven; only affected ranges
                           are re-enriched, with a before/after
                           diff retained
3 portfolio and BIN-range change events  issuer sale, reissue, range transfer
```

- | | | |
|---|------------------------|---|
| 4 | observed contradiction | a network response that disagrees with our stored attribute invalidates it and schedules re-enrichment. This is the channel that catches what the other three miss. |
| 5 | first-use verification | a credential vaulted before the current snapshot is verified out of band on first authorization; the transaction proceeds on the cached profile |

Why there is no on-path validation call

A validation that must complete inside the authorization budget is not a validation — it is a lookup with a deadline, and under load it degrades to whatever the cache already held. Rather than pay latency for a check we would ignore on timeout, freshness is carried as an attribute and consumed as a decision input. Where an attribute is stale and the decision depends on it, the platform constrains the decision (§12) rather than guessing.

Credential lifecycle



Network token lifecycle tracked separately and joined by reference:
PROVISIONING · ACTIVE · SUSPENDED · RESUMED · DELETED · UPDATED

Forward API

The vault's competitive position rests on credentials being portable rather than captive. Forward API lets an enterprise merchant execute against a credential held in our vault through a processor that is not us — the anti-lock-in argument that Adyen structurally cannot make. Architecturally this means the vault must be usable without the orchestrator, so credential references are stable, independently authorised objects rather than internal handles.

Screening and financial crime

Absent from the reviewed design and from v2.2's module inventory. For a bank-operated PSP this is not optional, it is not asynchronous, and it consumes real latency budget.

Check	Placement	Budget	On failure
Sanctions — party screening	Onboarding + periodic rescreen	Off path	Account block
Sanctions — transaction screening	Auth path, parallel node	≤ 12 ms p99	Hard fail, no fallback
Cross-border / country risk	Auth path, in constraint set	0 ms (precomputed)	Constraint block
AML transaction monitoring	Post-authorization, event-driven	Off path	Case creation
Prohibited MCC / activity	Constraint set	0 ms (precomputed)	Constraint block

Design rule

Screening is the only module with no fallback behaviour. If it cannot reach a decision within budget, the transaction fails. Every other module degrades; this one does not, and the orchestrator's `HARD_FAIL` disposition exists specifically for it. Any future module requesting `HARD_FAIL` requires architecture review.

Platform security

Named as the largest omission in v3.3 and still absent in v3.4. A bank-owned PSP cannot take a target-state architecture to approval with security inherited by assumption. This section states the zones, the key hierarchy and the identity model; it does not replace a full security design, and §31 says so.

Cryptographic zones

ZONE 0

INTERNET

| TLS termination, DDoS, bot defence – no card data trust

▼

ZONE 1

API EDGE

| authn, idempotency, schema. Never sees a PAN except in transit to the ingress boundary

| non-CDE

▼

ZONE 2

PAN INGRESS BOUNDARY

| the only place a raw PAN may enter. Client-side encryption or hosted fields terminate here

| CDE entry

▼

ZONE 3

CREDENTIAL VAULT

| tokenisation, credential storage, network token provisioning. Emits references only (§08)

| CDE

▼

ZONE 4

HSM / KEY BOUNDARY

| key generation, storage, P2PE, PIN block translation. No application code executes here

| CDE, highest

▼

ZONE 5

PROCESSOR ADAPTER

| detokenisation for network message construction only, inside the zone, never returned upward

| CDE egress

▼

ZONE 6

NETWORK

EVERYTHING ELSE – orchestrator, decision platform, learning, experimentation, money movement, reporting – is non-CDE and operates exclusively on credential references. That is the property that keeps PCI scope bounded as the platform grows; a single module that takes a PAN as input collapses it.

Key hierarchy and rotation

Root of trust

└─ HSM-resident, dual-control ceremony, offline

└─ Zone master keys

└─ per environment, per region

└─ Data encryption keys

└─ per credential class

└─ Working keys

└─ rotated on schedule and on event

Rotation

scheduled per key class; event-driven on personnel change, suspected compromise, or a failed attestation

rotation is exercised, not merely documented – an unexercised rotation is an untested rollback (§27)

Cryptographic erasure

a credential's destruction is the destruction of its key material, not a delete of a row. This is what makes the retention commitments in §26 enforceable

Identity, authorization and access

Concern	Mechanism
Service identity	Workload identity issued at deploy, short-lived, non-transferable; mTLS between every service, no network-position trust
Service authorization	Explicit per-call policy — which service may invoke which operation on which resource class. Default deny
Operator access	Named, time-bound, purpose-recorded. No standing production access to CDE zones
Break-glass	Dual control, pre-authorised scope, full session recording, mandatory post-review within 24 hours — same discipline as emergency constraint propagation (§06)
Secrets	Brokered at runtime, never in images, config bundles or plan artefacts. A signed plan bundle carries no secret
Log redaction	Enforced at serialisation, not by convention. A field that could carry a credential is typed such that it cannot be logged

Replay is a security boundary, not a tool

The replay engine (§22) can re-execute any historical transaction. That is the platform's most powerful diagnostic capability and its most attractive target: an attacker with replay access has an oracle over historical payment behaviour. Replay therefore runs as its own zone with CDE-equivalent controls, credential material never leaves it, every replay is attributed to a named operator, and replay of another operator's traffic requires the same dual control as break-glass. Treating replay as an ops feature rather than a security zone would be the single most likely way this architecture is compromised.

Tenant separation as a security property

Cross-tenant data exposure in a platform serving PayFacs and submerchants is a breach, not a bug. Authorization is evaluated against the client hierarchy (§05) on every access, including internal service calls and including replay and analytics paths, where cross-tenant queries are otherwise easiest to write by accident.

Authentication

Authentication is a decision before it is a protocol. The wrong model is a merchant flag; the right model is a strategy resolved from constraint, risk, economics and preference, in that order.

Authentication strategy resolution

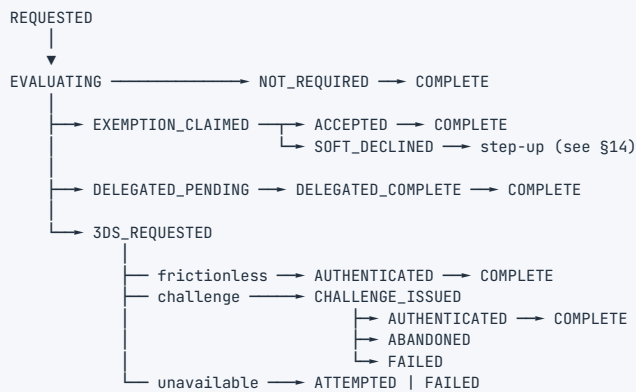
REGULATORY CONSTRAINT	is SCA applicable? which exemptions exist? (from constraint set – already resolved)
↓	
SCHEME REQUIREMENT	mandates, data requirements
↓	
INSTITUTIONAL RISK	JPM appetite for this segment
↓	
RISK SIGNAL	fraud score, device, velocity, history
↓	
ECONOMICS	liability shift value vs conversion cost vs interchange effect
↓	
MERCHANT PREFERENCE	within what the above permit
↓	
LEARNED STRATEGY	choose among permitted options

Decision surface

The output is not a boolean. It is a selection among authentication actions with an expected-value basis, recorded so the choice can be evaluated later.

```
AuthenticationDecision
├── action      NONE | EXEMPTION_REQUEST | 3DS_FRICTIONLESS
│              | 3DS_CHALLENGE | DELEGATED | PASSKEY | WALLET_INHERITED
├── exemption_type  TRA | LOW_VALUE | RECURRING | MIT | SCP | TRUSTED
├── basis           CONSTRAINT | ECONOMICS | PREFERENCE | LEARNED
├── expected_effect { auth_rate_delta, conversion_delta,
│                  interchange_delta, liability_shift }
└── fallback_if_unavailable
```

Authentication state machine



COMPLETE carries the evidence bundle forward to every subsequent attempt on the same intent – authentication is bound to the intent, not the request, so a processor substitution does not re-authenticate.

Authentication context

```
AuthenticationContext
├── protocol_version · ECI · CAVV/AAV · authentication_value
├── ds_transaction_id · acs_transaction_id · three_ds_server_id
├── method · challenge_indicator · challenge_result
├── exemption_claimed · exemption_accepted · exemption_reason
├── delegated_authentication { provider, assurance_level }
├── device { id, fingerprint, browser, os, app }
├── network_signals { issuer_authentication_data, trusted_listing }
├── evidence_retained_until (retention policy, §26)
└── outcome_liability { shifted: bool, basis: "..."}

```

Change from v2.2

Authentication context binds to the Payment Intent rather than the Payment Request. This is what makes step-up-then-retry and processor substitution work without re-challenging the cardholder, and it is a direct consequence of introducing the Intent/Request split in §04.

Routing and debit

Routing selects a processor, an acquiring relationship, a network and a message format. In the US debit context that last choice is regulated, and treating it as an optimization variable without a constraint gate is a compliance failure, not a performance opportunity.

Routing decision inputs

```
RoutingDecision
  inputs  constraint_set · instrument attributes (vault-time) ·
          risk_score · credential_selection · merchant_config ·
          processor_health · cost_table · historical_performance
  output  { processor, acquirer, network, message_format,
            merchant_account, reason_code, regime, propensity }

  objective (when regime = LEARNED, within constraint bounds)
    maximise E[approval] · value
    - E[cost_of_acceptance]
    - E[operational_risk]
```

US debit and Regulation II

Specified explicitly because it is where our BIN intelligence investment converts into a defensible capability, and because getting it wrong is a regulatory matter rather than a revenue matter.

Element	Treatment
Network availability	Resolved at vault time from the PIH debit overlay; at least two unaffiliated networks must be enabled where the issuer supports them
Merchant routing choice	Merchant preference plane; the merchant’s right to choose is a constraint, not a setting we may optimise away
PINless eligibility	Versioned pure function over BIN snapshot and merchant configuration; reproducible for audit
Least-cost routing	Learned strategy operating only over networks the constraint set has declared permissible
Single vs dual message	Explicit output of routing, not an adapter implementation detail — it changes reversal, capture and dispute semantics downstream
Signature vs PIN vs PINless	Selected as part of the route, carried in the message format decision

Message formatting is a decision, not a template

The same authorization, correctly formatted for two issuers, is not the same message. Field population, descriptor construction, address formatting and enrichment presence all affect approval, and they affect it differently by issuer, network and country. Formatting is therefore an output of Message Intelligence with a version and a measurable effect, not a static mapping inside the adapter.

```
Dynamic formatting dimensions
  descriptor construction  length, prefix, sub-merchant handling
  address and AVS presentation  formats that pass in one country
                              and fail verification in another
  enrichment field selection  which optional fields to populate;
                              more is not always better – some
                              issuers decline on unexpected fields
  indicator selection        COF, MIT/CIT, recurring, instalment
  amount presentation        partial-auth and incremental support

Properties
  versioned per (issuer or BIN range) × network × channel
  learned within the constraint set, exactly like routing
  A/B testable as its own POEM layer (§18)
  the adapter renders; it does not decide

Interchange qualification effects of formatting are attributed to
this module so its value shows up in the pricing model (§19) rather
than disappearing into routing's numbers.
```

Posture when instrument attributes are stale

Staleness changes what routing is permitted to optimise. It does not change the message model, and it does not silently pick a different rail — both would be worse than the problem.

Attribute state	Routing posture	Recorded as
Debit network participation fresh	Full least-cost optimization within permitted networks	regime=LEARNED
Participation stale or unverified	Least-cost optimization suppressed; route to merchant-configured default network	regime=FALLBACK, reason=STALE_ATTRIBUTE
Participation absent entirely	Route to configured default; flag for enrichment; no inference from the BIN range	regime=FALLBACK, reason=MISSING_ATTRIBUTE
Regulated status uncertain	Treat as regulated for compliance purposes; do not claim exemption	regime=POLICY
Network response contradicts stored attribute	Complete the transaction; invalidate attribute; re-enrich; the contradiction is a learning signal	Invalidation event

A rejected remedy

Falling back to a single-message rail when overlay data is absent was proposed and is not adopted. Message model is a property of the merchant's capture and reversal flow, not a safety valve — switching it changes settlement and dispute semantics downstream, and it does not achieve Reg II compliance in any case. The correct conservative action is to stop optimising, not to change rails.

Adapter boundary

Network-specific complexity terminates at the adapter. But the design admits the leak rather than claiming perfect abstraction: message format, reversal semantics and partial-authorization support differ enough that the canonical model carries them as first-class attributes.

Canonical instruction
↓
Processor adapter (inside PCI boundary)
↓
ISO 8583 · network API · scheme-specific message

Adapter declares, per processor and network:

message_model	SINGLE DUAL
supports_partial_auth	bool
supports_incremental	bool
reversal_semantics	AUTO_REVERSAL EXPLICIT NONE
timeout_disposition	what an unanswered request means here
stand_in_behaviour	issuer/network stand-in characteristics

These attributes flow back into Recovery (§15) and Attempt Strategy (§14).
An adapter that does not declare them cannot be enabled for production.

Rail execution model

v3.3 admitted this specification was card-shaped and gave non-card rails an owner without a design. Forcing authorize-then-capture onto rails that do not work that way is how a card PSP fails at everything else, so the execution model generalises here rather than the rails being made to conform.

Rails have different shapes

CARD	Intent → Authorization → Capture → Clearing → Settlement reversible before capture; disputable after
ACH	Intent → Submission → Acceptance → Return window → Settlement no authorization; risk lives in the return window, which can be 60 days
RTP / FedNow	Intent → Authentication → Push → Final settlement irreversible on receipt. There is no reversal, only a request for return that the receiver may decline
OPEN BANKING	Consent → Authentication → Payment instruction → Settlement authentication precedes and authorises the payment; it is not a step inside it
WIRE	Intent → Instruction → Pending → Settlement recall is a manual, negotiated process
AFT / OCT	Funding instruction → Network processing → Settlement push to card; no capture, different dispute rights
WALLET	Intent → Wallet authorisation → Underlying rail inherits the shape of whatever funds it

The common abstraction

PaymentIntent	unchanged – rail-agnostic
↓	
PaymentRequest	gains rail_type and rail_operation
↓	
RailExecution	rail-specific state machine, declared by the rail adapter, not assumed
↓	
RailOutcome	normalised into the canonical result vocabulary (§23)
↓	
MoneyMovement	§19, rail-aware

Every rail adapter declares, and cannot be enabled without:

states[]	its own lifecycle
reversibility	REVERSIBLE_UNTIL(state) IRREVERSIBLE
finality_point	the state after which money is gone
return_window	duration and semantics, if any
authentication_position	INSIDE PRECEDING NONE
recovery_actions[]	what is even possible when it fails
dispute_rights	who can dispute, on what basis, for how long

What this changes elsewhere

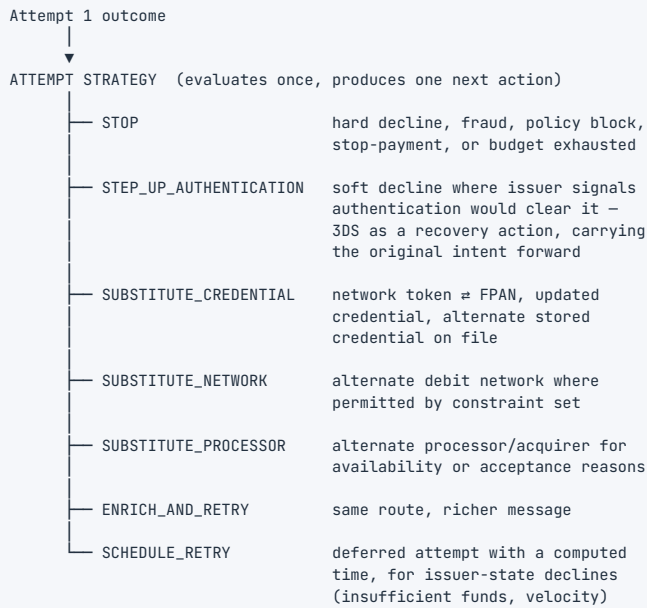
Section	Consequence
\$14 Attempt strategy	Retry is meaningless on an irreversible rail after the finality point. Substitution across rails changes the customer's agreement and is a new decision, not a retry
\$15 Recovery	Compensation is rail-declared. On RTP there is no reversal to issue — recovery becomes a return request with an uncertain outcome, and the supervisor must model that as a distinct case type
\$11 Authentication	Where authentication precedes the payment, the authentication context is created before the intent and attaches to it, rather than being produced during execution
\$19 Money movement	ACH return windows create a liability that persists after settlement; the ledger holds a return-risk position that card volume does not generate
\$16 Learning	Outcome horizons differ by rail. An ACH outcome is not final for weeks, so short-horizon promotion (§16) is unavailable there

Honest scope

This section makes the execution model rail-agnostic. It does not design the individual rails: ACH return handling, RTP return requests, open banking consent lifecycle and wallet delegation each need their own design pass. What changes here is that the platform can now hold them without distortion, which it could not before.

Attempt strategy

One domain owns the question "what do we try next." Merging retry and authentication step-up into a single decision surface is the most consequential change from v2.2, because it is where a measurable share of recoverable declines currently escape.



Termination — why this cannot loop

Merging step-up authentication with substitution creates a graph, and graphs cycle. Three independent mechanisms guarantee termination, and none of them relies on the others being correct.

- 1 MONOTONIC PROGRESS
Every attempt is characterised by a substitution vector:

```
{ credential, network, processor, authentication_state,
  message_enrichment, amount }
```

A new attempt must differ from every prior attempt on this request in at least one dimension. The visited set is held on the request and is part of its state. An action that would reproduce a visited vector is not offered.
 - 2 TYPED BUDGETS
Budgets are per dimension, not one counter, so exhausting processor substitution does not consume the authentication allowance:

step_up_attempts	≤ 1	per intent
credential_subs	≤ 2	
network_subs	≤ 1	
processor_subs	≤ 2	
scheduled_retries	per scheme rule, per merchant policy	
total_attempts	hard ceiling, always lowest binding	
 - 3 HARD CEILING AND ESCAPE
An absolute per-request attempt ceiling and an absolute per-request wall-clock deadline, both enforced by the orchestrator rather than by the strategy module. On breach:
HARD_STOP → canonical failure to the merchant, and
SUPERVISOR_CASE → Recovery (§15) resolves any open exposure
- Cross-border note: substitution across acquiring boundaries changes the transaction's regulatory character. A substitution that would alter SCA applicability, interchange treatment or the acquiring jurisdiction is not a retry – it is a new decision, and it is re-evaluated against the constraint set rather than inheriting the original evaluation.

Inputs and bounds

The strategy consumes the canonical decline classification (§23), the network and processor behaviour attributes, the credential state, the authentication state, and the constraint set's retry bounds. It is never permitted to exceed the bound, and every action is attributed on the resulting

attempt via `strategy_origin` so its effect is measurable.

Bounded by constraint set:

max_attempts_per_request

scheme and issuer rules

max_attempts_per_period

scheme excessive-retry rules

min_interval_between_attempts

permitted_actions[]

merchant may disable step-up

cost_ceiling

do not spend more recovering
than the transaction is worth

Never permitted:

blind repetition of an identical failed attempt

retry after a hard decline, fraud decline or stop-payment

step-up where the constraint set prohibits authentication

substitution that would change the cardholder's agreement

`Gap closed`

The reviewed design has a retry engine (its §36) and an authentication engine (its §11–11) with no path between them. Neither can express "this soft decline would clear with authentication." That path is a meaningful share of recoverable volume in card-not-present, and it is the specific gap the Stripe whitepaper pressure test surfaced against v2.1.

Recovery and compensation

The unowned domain. Both the reviewed design and v2.2 assert that the orchestrator contains no business logic and that modules are stateless — and then leave unstated who resolves a timeout with an unknown outcome. That gap is where real money is lost.

The unknown-outcome problem

```
authorize dispatched → processor → network → issuer
                        |
                        no response within deadline
                        |
                        may have
                        approved,
                        declined,
                        or never
                        arrived
```

The orchestrator cannot decide what this means: the answer depends on the adapter's declared `timeout_disposition`, the network's stand-in behaviour, the message model, and the merchant's exposure tolerance. The module cannot decide: it is stateless and already gone.

Therefore: a named supervisor owns it.

Recovery supervisor

Trigger	Resolution path	Deadline
Authorization timeout, unknown outcome	Reversal advice, then status inquiry, then reconciliation against network clearing	Immediate → 24h → settlement window
Approved authorization, capture failed	Retry capture within window; if impossible, reversal	Capture window
Partial success in multi-leg order	Order-level compensation policy: hold, refund collected legs, or continue	Order expiry
Authorization succeeded, downstream state write failed	Journal reconciliation; state rebuilt from events	Seconds
Orphan authorization found at clearing	Attach to intent if identifiable; otherwise exception case	Settlement
Duplicate authorization detected	Reverse the later; alert if captured	Immediate
Step-up authentication completed but original request expired	New request under same intent, authentication evidence carried	Auth validity window

Compensation in the plan

Every plan declares its compensation graph alongside its execution graph. Compensation is planned, versioned and testable rather than improvised during an incident.

```
"compensation": {
  "authorize": {
    "on_timeout": ["reversal_advice", "status_inquiry", "clearing_reconcile"],
    "on_unknown": ["hold_intent", "supervisor_case"],
    "on_late_approval_after_abandonment": ["auto_reversal", "notify_merchant"]
  },
  "capture": {
    "on_failure": ["retry_within_window", "reversal", "supervisor_case"]
  },
  "order_leg_failure": {
    "policy_ref": "order.compensation_policy"    // merchant-configurable
  }
}
```

Ownership

Recovery is a domain with an on-call owner, an error budget and a reconciliation SLA measured against the settlement window. Its primary metric is unresolved-outcome age, not incident count. If no team owns this, it will be owned by the support organisation as a queue of manual cases — which is what the current stack does.

Learning layer

The reason this platform can close a decade of competitor learning curve, and the part of the design that is hardest to get past model risk. Both facts are structural: the mechanism that makes learning fast is the same mechanism that makes it governable, because both depend on every decision being logged with its propensity and its counterfactual alternatives.

Autonomy ladder

Level	Behaviour	Human role	Governance
0 — Fixed	Static configuration	Sets everything	Change control
1 — Measured	Static, but counterfactuals logged	Sets everything	Change control
2 — Advisory	Model recommends; humans act	Approves each change	Model validated, not in production decision
3 — Bounded auto	Model decides within human-set bounds	Sets bounds, reviews weekly	Full SR 11-7 scope
4 — Auto with veto	Model decides and adapts bounds	Reviews exceptions	Full scope + drift monitoring
5 — Autonomous	Model sets objectives within policy	Audits outcomes	Not proposed for any decision point in this design

Autonomy is set per decision point *and* per client. A strategic enterprise merchant may run routing at level 2 while the long tail runs at level 3. The platform must be genuinely good at level 0, because that is where every new client starts and where enterprise accounts may permanently sit.

POEM — experimentation at two scales

MACRO

plan versioning (§07)
whole execution paths compared
slow, safe, coarse
answers "is plan v43 better than v41"

MICRO

POEM cells at individual decision points
orthogonal assignment across decision points so that
routing, retry and authentication experiments do not
confound each other
fast, targeted, fine-grained
answers "does routing arm B beat arm A, holding
authentication strategy constant"

Assignment happens once, at context assembly, and is carried in the
context so every decision point sees the same assignment vector and
logs it identically.

Decision record — the training substrate

Every learned decision logs what was chosen, what else was available, and with what probability it was chosen. Without propensity, off-policy evaluation is not possible and the platform can only learn from live experiments — which is the slow path we cannot afford.

DecisionRecord

— decision_point

— context_features

— action_taken

— action_space

— propensity

— policy_version

— model_artefact

— poem_cell

— outcome

— outcome_lag

routing | retry | authentication | credential

the features actually used, versioned schema

everything the constraint set permitted

P(action | context, policy_version)

the decision policy in force

versioned, immutable

joined asynchronously: approval, cost,
fraud, dispute, downstream conversion

some outcomes arrive in days (fraud),
some in months (dispute)

Off-policy evaluation

Candidate policy π' evaluated against logged data from π
without exposing a single live transaction

inverse propensity weighting

doubly robust estimation

clipped importance weights

→ unbiased but high variance

→ the working default

→ bounded variance

Promotion gate: a candidate policy must clear OPE on held-out logged data, then a bounded live POEM cell, then a review, before any traffic ramp. Three gates, none skippable.

Exploration

A system that only ever takes the action it currently believes is best stops learning, and its logs lose the overlap that off-policy evaluation depends on. Exploration is therefore a deliberate, budgeted, governed behaviour rather than an emergent property — and it was specified in v2.2 as contextual bandit optimization before being lost in consolidation.

Where exploration is permitted

routing	yes, within the permitted processor set
attempt strategy	yes, within permitted actions
message enrichment	yes
authentication	limited — an exploratory challenge costs a real cardholder a real interruption
credential selection	yes
screening, constraint	no, never

Form

contextual bandit over the constraint-bounded action set, with the propensity of every draw logged (\$16 decision record). The constraint set defines the arms; the bandit never widens it.

Budget

exploration_share is an entitlement-plane setting with a platform ceiling. A merchant may set it to zero; the platform must still function, and that merchant then benefits only from strategies learned on other traffic.

Expressed as expected cost, not as a percentage of transactions: "we will spend up to X basis points of approval rate learning" is a statement a commercial owner can actually approve.

Never explored

- high-value transactions above a configured ceiling
- first transaction for a new customer on a merchant
- transactions where a prior attempt already failed — recovery is not the place to gather evidence
- strategic accounts, unless separately agreed

Why exploration is a commercial conversation

Exploration has a cost denominated in declined transactions, and that cost lands on identifiable merchants. Framing it as a percentage of traffic hides this; framing it as basis points of approval rate makes it a decision someone can own. The alternative — exploring implicitly and calling the cost noise — is how a platform loses an account it cannot explain the loss of.

Start logging before the platform exists

The one thing that cannot wait for migration

Propensity and the counterfactual fallback cannot be reconstructed after the fact. Every month the legacy stack runs without emitting them is a month of traffic that will never be evaluable, and off-policy evaluation of the first candidate policies depends on having logs that predate the decision platform. Routing and retry decisions on the current engine can emit `{ considered[], action, propensity, fallback_would_have_been }` today, with no behaviour change — the decision is unchanged, only the record is richer. This is step 4 of the migration sequence (§28) and it is the cheapest high-value item in the programme.

Promotion gates

Three gates, each with a stated pass criterion. A candidate that clears all three still ramps under an exposure predicate; clearing the gates buys the right to be tried, not the right to be deployed.

Gate	Pass criterion	Failure mode it catches
1 — Log adequacy	Effective sample size above threshold in every segment the candidate would act differently in; overlap between behaviour and candidate policy support confirmed; clipping mass below threshold	An estimate computed over segments where the logs contain almost no evidence
2 — Off-policy estimate	Doubly robust estimate with a confidence interval that excludes harm, per objective term, in every affected segment — not merely a low variance figure	A candidate that is better on average and worse for an identifiable population
3 — Bounded live cell	POEM cell result directionally consistent with the OPE estimate, within the interval, at pre-registered sample size	Systematic estimator bias that no amount of held-out data reveals

Variance is the wrong criterion on its own

A low-variance estimate is not a trustworthy one. Variance falls when the behaviour policy was nearly deterministic — exactly the case where the logs contain no evidence about the alternatives, and where importance weights collapse. The gate is therefore stated as a confidence interval that excludes harm, supported by explicit overlap and effective-sample-size diagnostics. A candidate can fail gate 1 with a beautifully tight variance number.

The objective function is a governed artefact

Multi-objective optimization requires an explicit objective, and it must be written down, versioned and validated like any other model component. Three properties are non-negotiable.

1 COMMENSURABLE UNITS

Terms are expressed in expected currency per transaction, not dimensionless weights. Dimensionless weights cannot be reasoned about, cannot be validated, and hide the exchange rate between approval and cost that is the whole point of the exercise.

$$E[\text{value}] = P(\text{approve}) \times \text{margin_on_approval} \\ - E[\text{cost_of_acceptance}] \\ - E[\text{expected_fraud_cost} \mid \text{routing choice}]$$

2 THE TRADEOFF IS THE MERCHANT'S

The exchange rate between approval and cost is a commercial preference, not a platform constant. A subscription business and a low-margin retailer are correctly at different points on that curve. It lives in the preference plane (§06), inside bounds set by the entitlement plane:

```
merchant.optimization_posture =
  MAXIMISE_APPROVAL | BALANCED | MINIMISE_COST | EXPLICIT_CURVE
```

3 CONSTRAINTS ARE NOT REWARD TERMS

Latency budget and risk appetite are constraints. Expressing them as penalties lets the model trade them away when the payoff is large enough, which is precisely what a constraint exists to prevent. The deadline is enforced by the orchestrator; risk appetite is enforced by the constraint set; neither is negotiable by the objective.

Governance

the objective function is an artefact with a version, an owner and a validation record; a weight change is a model change requiring revalidation, not a configuration edit

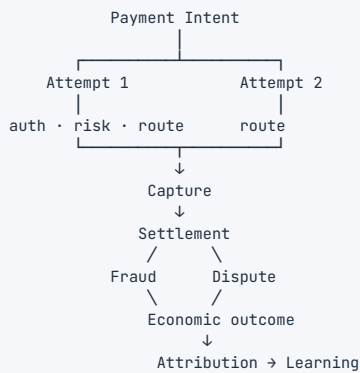
Economic decision policy — three layers

- HARD CONSTRAINTS** never violated, never traded
regulatory · scheme · screening · institutional risk appetite
· latency deadline
- ECONOMIC CONSTRAINTS** bounds within which optimisation operates
max cost per transaction · max processor concentration
· max expected fraud cost · max FX exposure · capital cost
· merchant margin floor
- OBJECTIVE** maximised inside 1 and 2
 $E[\text{economic value}] = E[\text{revenue on approval}]$
- $E[\text{processing cost}]$
- $E[\text{fraud loss}]$
- $E[\text{dispute loss}]$
- $E[\text{operational cost}]$
- $E[\text{capital and FX cost}]$

Concentration is an economic constraint rather than an objective term for a reason: an optimizer that can trade concentration against approval will eventually route a dangerous share of volume to one processor, and will be locally correct every time it does.

The outcome graph

A strategy is evaluated against the whole downstream economic outcome, not against approval. This is the guard against the classic optimisation failure: approval up forty basis points, fraud and disputes up more than that, discovered two quarters later.



Every decision node links forward to every economic consequence. No strategy is promoted on an approval metric alone; the promotion gate requires the long-horizon terms to be at least monitored, and a strategy promoted on short-horizon evidence carries a scheduled review at which they are assessed (§16, horizons).

Causal attribution across concurrent decisions

With several layers live at once, the question "which change produced the improvement" is genuinely hard, and it is worth being precise about what is and is not estimable.

Question	Estimable?	How
Effect of one arm against its own control, within its layer	Yes	Randomised assignment, standard
Interaction between two concurrently live layers	Yes, if both are randomised	Factorial analysis across layers — this is why orthogonal salts exist (§18)
Decomposition of an aggregate movement across decisions that were not all randomised	No	Not identifiable. Reported as unattributed, not apportioned by assumption
Contribution of a strategy against its declared fallback	Yes, with an interval	Propensity-weighted, per §17

A promise not made

A clean decomposition of the form "18 bps from routing, 11 from credential selection, 9 from authentication, 4 interaction" is available only when every one of those decisions was randomised in the same period. Where they were not, the honest output is a main effect for what was randomised and an explicitly unattributed remainder. Apportioning that remainder by plausibility would produce numbers the organisation would then plan against, which is worse than reporting that we do not know.

Outcome horizon and credit assignment

Outcomes arrive at very different lags, so a single scalar reward is under-specified until the horizon is stated.

approval / decline	seconds
capture and settlement	hours to days
fraud	days to weeks
dispute	weeks to months

Rules

- each objective term declares its outcome horizon
- early promotion may use short-horizon terms only, with the long-horizon terms monitored and able to trigger demotion
- late-arriving outcomes update the evaluation of decisions already made — retrospective revision of a policy's measured

performance is expected, not an anomaly
a policy promoted on short-horizon evidence carries an explicit
review date at which long-horizon evidence is assessed

Reward integrity

An optimizing system with an externally influenced reward signal is an attackable system. The threat is not hypothetical: a processor whose response behaviour affects our measured performance has an incentive to influence it.

Tests run before promotion and continuously in production

poisoned-signal injection	synthetic corrupted outcome streams; the constraint layer must hold and the anomaly must be detected, not absorbed
reward drift monitoring	distribution shift in outcome signals independent of feature drift
counterparty influence	a dependency's response characteristics changing our measured reward without a corresponding change in merchant outcome
feedback loop detection	a policy whose own actions change the data it will later learn from – routing starves an acquirer, then concludes the acquirer is unproven

On detection: automatic autonomy downgrade to the next lower level
(§16 ladder), not an alert for someone to triage.

Risk is a vector, not a score

A single fraud score forces every downstream decision to consume one number that was trained against one outcome. The decisions differ, so the predictions must too — another v2.2 finding that did not survive consolidation into 3.0.

RiskPrediction

- p_fraud third-party fraud
- p_dispute dispute regardless of fraud – these are
 different populations and only one of
 them is a loss we can contest
- p_false_decline the cost nobody logs: a good transaction
 this decision would lose
- p_abuse promo, refund and policy abuse
- p_issuer_decline predicted issuer behaviour, which is a
 routing input rather than a risk input
- each with its own model artefact, horizon and confidence

Consumers take what they need

authentication (§11)	p_fraud and p_false_decline together – step-up is a conversion cost, so the tradeoff needs both sides
routing (§12)	p_issuer_decline
attempt strategy (§14)	p_issuer_decline and p_dispute
constraint set (§06)	p_fraud against risk appetite

The single-score design silently forces every consumer to treat
false declines as free, because the score never measured them.

Model portfolio, ordered by feedback speed

Model	Feedback lag	Priority	Autonomy target
Processor availability prediction	Seconds	First	3
Approval likelihood by route	Seconds	First	3
Retry timing and action selection	Minutes to days	Second	3
Authentication step-up value	Minutes	Second	2 → 3
Credential selection (token vs PAN)	Seconds	Second	3
Message enrichment value	Seconds	Third	3
False-decline prediction	Minutes	Second	2
Fraud scoring	Days to weeks	Third	2
Abuse prediction	Days	Third	2
Dispute likelihood	Weeks to months	Last	2

Build order follows feedback speed, not perceived business value. A model whose outcome arrives in seconds can be improved weekly; a model whose outcome arrives in months can be improved twice a year, and its errors compound silently in between.

SR 11-7 alignment

Requirement	Mechanism in this design
Model inventory	Every artefact versioned and referenced in module execution records
Conceptual soundness	Documented per decision point, reviewed at promotion gate three
Independent validation	OPE on held-out logged data by a party separate from the developing team
Ongoing monitoring	Drift on features and propensity distribution; automatic autonomy downgrade on breach
Outcomes analysis	Decision records joined to realised outcomes, including late-arriving ones
Limits and overrides	Constraint set is the limit; authority ladder makes overrides governed and logged
Documentation	Generated from the artefact registry rather than maintained separately

Honest constraint

Model governance is a real cost, not a checkbox. It adds weeks to every promotion and it will make us slower than Stripe at shipping a routing change. The compensating advantage is that our learned strategies are defensible to enterprise procurement and to regulators, which is a durable commercial asset in the segment we are targeting. Anyone proposing to shortcut this should be asked which segment we would then be selling to.

Decision platform

§16 specifies how the platform learns. This section specifies how a decision strategy is authored, evaluated, deployed, attributed and retired — which is a different problem, and one the specification previously answered by implication: ship a module version. That makes every product judgment an engineering release, and it is the single largest constraint on how fast acceptance rate and cost of acceptance can improve.

The decision point abstraction

Every decision the platform makes is the same shape. Formalising that shape is what lets new decision domains be added without touching the orchestrator — capture strategy, FX strategy, funding optimisation and recovery prioritisation all become decision points rather than new subsystems.

```
DecisionPoint
├── decision_point_id · domain · owner
├── input_schema      the context subset it may read
├── action_schema     the shape of a decision
├── constraint_binding how ActionSpace is computed (§06)
├── objective_ref     what it optimises (§16)
├── eligible_strategies · fallback_strategy
├── experiment_policy layer, unit of randomisation (§18)
├── autonomy_level    per client (§16 ladder)
├── attribution_model  how its contribution is measured
├── latency_budget    hard, owned (§26)

Uniform pipeline for every one of them

Context → ActionSpace → Strategy → Experiment assignment
        → Decision → Execution → Outcome → Attribution → Learning

Current decision points: authentication, credential selection,
routing, network selection, message formatting, attempt strategy,
capture strategy, FX strategy, recovery prioritisation. Adding a
tenth requires a registration, not an architecture change.
```

Flow — the named unit of execution

An execution plan with a version number is correct and unreadable. `exe_01JQ7F3M2K@42` tells an operator nothing, and the thing said out loud in a review should be the thing recorded on the transaction. A **Flow** is the human-readable binding of a plan: a complete, named, versioned way of processing one transaction type, with exactly one strategy bound at every decision point.

```
Flow
├── flow_id          Flow_2_MsgAugDropCVV_v1
├── txn_type         US_ECOM_CREDIT_CIT
├── role            BASELINE | VARIANT
├── baseline_ref     required when role = VARIANT
├── bindings[]      { decision_point, strategy_id }
├── diff_count      one per decision point, no gaps permitted
├── differs_at      computed against baseline_ref
├── plan_ref        the decision point that differs
├── state · expiry  the opaque execution plan it compiles to

TxnType
├── txn_type_id     US_ECOM_CREDIT_CIT
├── predicate       channel · funding · region · mandate
├── baseline_flow_id exactly one, always
├── eligible_flows[]

Naming, enforced at registration
TXN_TYPE  REGION_CHANNEL_FUNDING_MANDATE
STRATEGY  DecisionPoint_WhatItDoes_vN
FLOW     Flow_N_Descriptor_vN
EXPERIMENT EXP_Variant_vs_Baseline

Sequential names — Strategy1, Strategy2 — are rejected. The name
appears in the readout, the ticket and the incident, and a number
carries no meaning six months later.
```

Single-difference construction

A variant Flow must differ from its baseline at **exactly one** decision point. `diff_count = 1` is checked by the compiler at registration and a variant failing it does not compile. This is what reconciles flow-level traffic allocation with per-decision attribution: if the flows differ by one binding, any outcome difference is attributable to that binding without further argument. Change two things and you have measured a bundle — and if it wins you cannot tell which half to keep, or whether one half is negative and being carried by the other.

Strategy as the unit of change

The substrate already exists. Constraint-bounded action sets (§06), versioned artefacts with determinism snapshots (§22), propensity logging and promotion gates (§16), layered cells (§18), per-client autonomy, predicate exposure (§27). What is missing is a first-class object that a person authors and the platform deploys without a release cycle.

Unit	What it is	Changes via	Owner
Module	The runtime that executes strategies at a decision point	Engineering release	Capability team
Model artefact	A trained function producing predictions	Training and validation pipeline	Data science
Strategy	How a decision is made at a decision point, for a population	Authoring and promotion workflow	Product, ops, or the client
Constraint	What is permitted at all	Control plane (§06)	Compliance, risk, scheme

```
Strategy
├── strategy_id · version · owner · decision_point
├── class          DETERMINISTIC | PARAMETRIC | LEARNED
├── population     exposure predicate – the population it may act on
├── objective_ref  whose objective it optimises (§16); for a
                  merchant-scoped strategy this is that merchant's
                  posture, not a platform average
├── definition     rule set, parameter set, or policy artefact
├── fallback      the strategy that applies if this one cannot
                  produce a decision – never an implicit default
├── governance    validation record, approval chain, review date
├── lifecycle     DRAFT → BACKTESTED → EVALUATED → PILOT
                  → ACTIVE → SUPERSEDED | RETIRED
├── expiry        mandatory; a strategy with no review date
                  becomes an unowned rule nobody dares delete
```

Constraint-bounded by construction

The property that makes this safe

A strategy must be unable to express an action outside the permitted set, not merely rejected when it tries. If non-compliance is caught by validation, the authority ladder is advisory and every new authoring surface is a new way around it. The action set a strategy selects from is *derived* from the resolved constraint set at runtime, so a strategy names preferences over an action space it does not control.

```
What a strategy expresses
  ordering and weighting over the permitted action set
  conditions under which it declines to decide (falls through)
  parameters within governed bounds

What a strategy cannot express – not "must not", cannot
  an action absent from the resolved constraint set
  a change to the constraint set
  a change to another strategy's population
  suppression of screening, journalling, or propensity logging
  its own exposure level
```

Three classes, three governance treatments

Applying model risk governance to a deterministic merchant rule would kill the self-serve path before launch; applying nothing to a learned policy would be a finding. The classes exist to make that distinction defensible rather than discretionary.

Class	Definition	Governance	Max autonomy
Deterministic	Rules with human-set conditions and actions; no fitted parameters	Change control, backtest, approval; not a model	Applies as written
Parametric	Human-specified form, parameters fitted from data	Full SR 11-7 scope for the fitting; form is reviewable by inspection	Level 3
Learned	Policy learned end-to-end from logged decisions	Full SR 11-7, plus OPE gates and drift monitoring (§16)	Level 3, level 4 by exception

The boundary is fitted parameters, not sophistication. A merchant writing "prefer network token over PAN for recurring" is authoring a deterministic strategy and should wait days, not quarters. The same merchant asking the platform to learn when to prefer it is asking for a model, and inherits the timeline that comes with one.

Strategy compiler

v3.4 asserted that a non-compliant strategy must be inexpressible rather than rejected. The compiler is the mechanism that makes that true: a strategy is never executable code, it is a source artefact compiled against the action space into a signed bundle.

Strategy source (DSL)
↓
Normalise
↓
Bind to ActionSpace (§06 algebra) ← the compiler knows what is permitted, statically
↓
Static validation
 ↓ "can this strategy ever select a forbidden action?"
 ↓ "can it reference context it is not entitled to read?"
 ↓ "does every path terminate in a decision or a fallback?"
 ↓ If yes to the first two, or no to the third: COMPILE FAILS
Objective binding
↓
Signed strategy bundle → registry → exposure

Example source

decision_point: routing
when: { country: US, channel: ecommerce, funding: credit }
objective: maximise expected_economic_value
subject_to:
 - cost_per_transaction < 0.07
 - processor_concentration < 0.40
prefer: [processor_a, processor_b]
fallback: routing_v17

No arbitrary code executes on the payment path – client-authored or internal. A DSL that compiles is auditable; a sandbox that executes is a permanent attack surface and an unbounded latency risk.

Registration gates

Four checks at registration, all mechanical. Each exists because the corresponding failure is invisible until the data is already unusable.

Gate	Rejects
Action-space binding	A strategy that could select an action outside the resolved action space
Single difference	A variant Flow with <code>diff_count ≠ 1</code>
Contract completeness	A decision point that cannot populate <code>propensity</code> and <code>fallback_would_have_been</code> in its response (§25). Both are mandatory fields, not conventions
Experimentability	A variant Flow differing at a decision point marked <code>experimentable: false</code>

DecisionPoint gains
 experimentable true | false
 false for: screening · constraint enforcement · anything
 altering amount, currency, or a disclosure flow

The screening RESULT remains available as a feature to downstream decisions. It is the decision that is not experimentable, not the data.

Evaluation workflow

Backtest, off-policy estimate and bounded live cell already exist as separate capabilities owned by separate teams. The decision platform makes them one workflow available to the strategy author.

- | | | |
|---|-----------|---|
| 1 | BACKTEST | replay against logged traffic (§22), shadow only, never dispatched. Output: decision diff against the incumbent, by category and population. |
| 2 | ESTIMATE | OPE with the §16 gates – log adequacy, confidence interval excluding harm per affected segment. Deterministic strategies skip this only where the diff shows no behaviour change; a rule that changes nothing needs no estimate, and one that changes something does. |
| 3 | PILOT | a bounded cell (§18), pre-registered, guardrails armed, at the strategy's own unit of randomisation. |
| 4 | RAMP | predicate exposure (§27), same control surface as plan versions and migration slices. |
| 5 | ATTRIBUTE | continuous, per strategy, in the objective's units. |
- Rollback at any stage is an exposure predicate change: seconds.

Attribution

The hard part, and the part most decision platforms quietly skip. Without it a strategy cannot be defended, sold, or retired, and the portfolio accumulates rules nobody can justify removing.

Method
every decision logs the action taken, the action the fallback strategy would have taken, and the propensity (§16). Contribution is the estimated difference against the fallback over the strategy's own population, in the objective's currency units.

Reported per strategy
transactions decided · approval delta · cost delta
fraud and dispute delta at their own horizons
confidence interval, always – a point estimate invites a precision the data does not support

Honest limits
overlapping strategies acting on the same transaction cannot be attributed independently; the platform reports joint contribution and names the overlap rather than dividing it by assumption
attribution is against the declared fallback, not against a counterfactual world with no strategy at all – the second is not estimable and claiming it would be dishonest

Contention

The layer registry (§18) permits one active cell per layer per population. Many concurrent strategies collide with that directly. Three rules keep the collision detectable rather than silent.

- | | | |
|---|---|--|
| 1 | ONE OWNER PER (decision point, population) | Populations are disjoint by construction. A strategy declares its predicate; the registry rejects a registration whose predicate intersects an active strategy's at the same decision point, and names the conflict. |
| 2 | ORDERED COMPOSITION WHERE OVERLAP IS INTENDED | Where overlap is deliberate – a platform-wide strategy plus a merchant refinement – composition order follows the authority ladder (§06), and the composed result is what is attributed, not each part. |
| 3 | CONTENTION IS VISIBLE | A strategy that never fires because a higher-authority one always precedes it is reported as inert, the same way an inert preference is (§06). Silent shadowing is how a strategy portfolio rots. |
- This resolves for internal authoring. It does not fully resolve for merchant self-serve at scale – see §31, item 15.

Exploration budget allocation

Exploration (§16) spends approval rate, and the spend lands on identifiable merchants. With one strategy that is manageable; with fifty competing for the same budget it needs an allocator.

Rule: the budget is scoped to the population that bears the cost. A merchant's exploration budget is denominated in basis points of that merchant's own approval rate, and no strategy may spend a population's budget to learn something that benefits only a different population.

Platform-funded exploration – where we deliberately spend our own economics to learn something we could not otherwise learn – is a separate, explicitly funded pool with a named owner. It is not drawn from merchant budgets by default.

Who allocates the platform pool is unresolved (§31, item 16).

Exploration budget manager

Exploration is capital allocation, and the decision platform asks permission to learn before it decides what to try.

ExplorationBudget

- scope merchant | platform pool | decision point
 - | geography | vertical | risk band
- denominated basis points of approval rate on the scope's own volume – never a percentage of transactions
- allocated · reserved · consumed · remaining
- max_loss_bps hard stop, enforced by the orchestrator
- owner · expiry
- priority how competing strategies are ranked when the budget is oversubscribed

Sequence at every learned decision

- 1 is there budget in this scope? if no → exploit only
- 2 is this transaction eligible? (§16 exclusions)
- 3 what should we try?

Merchant-scoped budgets are solved: a merchant spends its own approval rate, and no strategy may spend one population's budget to learn something benefiting another. The platform pool remains unowned – §31 item 16.

Authoring tiers

Who may author determines the blast radius, the support model and the governance load. These are three products, and shipping them in this order is a sequencing decision rather than a phasing convenience.

Tier	Author	UnLocks	Requires
1	Internal data science	Capability: strategies exist as artefacts at all	Registry, backtest, OPE gates, attribution
2	Internal product and operations	Speed: product judgment without a release cycle. The largest single gain	Above, plus a safe authoring surface and approval chain
3	Client, self-serve	A second product, with its own roadmap and support model	Above, plus contention at scale, per-client blast radius, and a support organisation that can explain a strategy's behaviour to its author

Sequencing

Tier 3 should not be promised until tier 2 is boring. A self-serve decisioning surface that produces behaviour its author cannot explain generates support load that scales with adoption, which is the worst shape a support cost can have. Tiers 1 and 2 are worth building regardless of whether tier 3 ever ships.

Internal strategy marketplace

Once strategies are registered artefacts with attributed economics, the registry becomes a marketplace: teams publish strategies with a hypothesis, evidence, current exposure and a promotion recommendation, and compete on measured economic contribution rather than on shipped features.

The incentive this creates, honestly

Rewarding teams on attributed basis points produces exactly the behaviour the metric describes, including its failure modes: population selection to flatter attribution, strategies scoped to the segments where measurement is easiest, and disputes over overlap that the attribution model cannot settle (§17, attribution limits). The marketplace is worth building for the transparency, not for the competition. If it is used for performance management, the attribution model becomes a compensation instrument and its honesty will not survive that.

Why this is a commercial asset, not only an internal one

The three objectives — acceptance rate, cost of acceptance, fraud — trade against each other, and §16 already establishes that the exchange rate between them is the merchant's to set. A decision platform makes that visible: strategies are evaluated against the merchant's own posture, and results are reported in their units rather than as a platform average.

That is a materially different claim from "our routing is good," and it is the version that is legible to the other-PSPs-and-banks segment, who are buying decisioning rather than acceptance. It also makes Forward API coherent: portable credentials plus portable decisioning is an anti-lock-in position Adyen structurally cannot offer. The caution is that a decisioning surface sold externally becomes a compatibility commitment — strategies clients depend on cannot be deprecated on our schedule.

Experimentation

POEM existed as a framework alongside UPOP, and its plumbing is specified across six sections of this document — assignment in §05, plan versioning in §07, cell identifiers in module execution records, the determinism snapshot in §22, disclosure in §24, traffic predicates in §27. What was never specified is the lifecycle: how an experiment is registered, at what unit it randomises, when it stops, and who kills it. A mechanism without a lifecycle becomes a drawer of forgotten cells.

Two scales, one control surface

MACRO

plan versions (§07)
whole execution paths compared; coarse, slow, safe
exposure: predicate over context (§27)
question: is plan v43 better than v41

MICRO

POEM cells at individual decision points
one decision point varied, everything else held
exposure: layered assignment (below)
question: does routing arm B beat arm A, holding authentication strategy constant

Both draw exposure from the same traffic planner. A transaction cannot be in a migration ramp slice and a micro cell at once – see precedence, below.

Registration

No cell is created without a registered record. The registry is queryable, and an unregistered assignment is a defect rather than a shortcut.

ExperimentRecord

- experiment_id · owner · decision_point
- hypothesisstated before launch, in one sentence
- primary_metricexactly one
- guardrail_metrics[]with abort thresholds
- unit_of_randomisationsee table below
- layerassignment layer and salt
- populationexposure predicate
- sample_sizepre-registered, with the effect size it is powered to detect
- outcome_horizoninherited from §16; disputes resolve in months, approval in seconds
- planned_end · hard_expiry
- decisionPROMOTE | REJECT | INCONCLUSIVE
 - recorded at end; an expired cell with no decision is terminated and logged as INCONCLUSIVE

Unit of randomisation

The most common way to invalidate a payments experiment is to randomise below the level at which the outcome is measured. Retry experiments randomised per attempt are the canonical error: the second attempt exists *because* of what the first arm did, so the arms are not independent.

Decision point	Unit	Why not lower
Attempt strategy / retry	Payment intent	Later attempts are caused by earlier ones; per-attempt assignment measures nothing
Authentication	Payment intent	Authentication context binds to the intent (§11) and survives retries
Routing	Payment intent, or customer where issuer-side learning effects are plausible	Issuer behaviour adapts to repeated exposure from the same cardholder
Credential selection	Credential	Token provisioning state persists across intents
Message enrichment	Payment intent	—
Plan version	Merchant, or merchant × customer	Merchants notice inconsistent behaviour within a session
Pricing or fee-affecting changes	Merchant	Charging two customers of one merchant differently is a commercial and contractual problem, not a statistical one

Assignment is a stable hash of (unit identifier, layer salt), computed once at context assembly and carried in the context. Never re-drawn on retry, never re-drawn on a new attempt, never drawn per request.

Concurrent experiments on one decision point

Two experiments allocating the same population at the same decision point do not compose — their allocations overlap and neither readout is usable. Three resolutions, chosen by whether the decision point is shared.

Situation	Resolution	Note
Same decision point	One experiment, three or more arms, shared control	Preferred. A shared control gives more power per unit of traffic than two separate 90/10 splits
Different decision points	Separate layers, independent salts	Assignment orthogonal by construction; interaction tested at readout
Same decision point, must be concurrent	Disjoint populations	Always safe; caps concurrency at the number of segments

ALLOCATION LEDGER — enforced at registration, not analysis

per (layer, segment):

holdout reserved off the top, always

$\sum$ arm allocations $\leq$ 1.0 - holdout

one active experiment ← uniqueness key

segment comes from a CONTROLLED VOCABULARY, not an arbitrary predicate. Detecting overlap between two free-form predicates is undecidable in the general case and expensive in practice; a fixed segment list makes the uniqueness check a lookup. Segments are added deliberately, by the same process that adds a txn type.

A second registration against an occupied (layer, segment) is rejected and names the conflict. By analysis time the traffic has already been double-allocated and no correction exists.

Permanent holdout

A fixed share of every population receives baseline Flows and is never assigned to anything. Two percent is the working default.

Why it cannot be added later

without it, individual experiments are measurable but the cumulative effect of the platform is not — and "what has all of this been worth" is the question leadership eventually asks. A holdout established retroactively measures nothing, because the population was already treated.

The holdout is excluded from exploration as well as from experiments, so it is a genuine baseline rather than a no-experiment-but-still-learning population.

Layers and orthogonality

v2.2 asserted orthogonal assignment. Assertion is not a mechanism, so here is the mechanism.

Each decision point sits in a numbered layer with its own hash salt. Independent salts make assignment across layers independent by construction, so a transaction can be in a routing cell and an authentication cell simultaneously without correlation between them.

layer 1 authentication salt_1

layer 2 routing salt_2

layer 3 attempt strategy salt_3

layer 4 message enrichment salt_4

assignment(layer, unit) = hash(unit_id || salt_layer) mod buckets

MUTUAL EXCLUSION

Two cells acting on the same decision point cannot both apply. The layer registry enforces one active cell per layer per population; a second registration against an occupied layer is rejected at registration time, not discovered in analysis.

INTERACTION DETECTION

Orthogonal assignment does not mean orthogonal effects. An

authentication arm changes the decline mix that a retry arm then operates on. Every readout therefore tests the interaction with concurrently live layers, and a significant interaction blocks promotion until both cells are analysed together.

Treatment fidelity

A transaction assigned to an arm may not receive its treatment: the decision point fell back, degraded, or breached its deadline. Both the naive responses are wrong.

Do not drop it dropping fallback cases biases the result toward periods when the platform was healthy, which is not the population the strategy will run in

Do not ignore it an arm with 4% fallback and one with 0.1% are not the same experiment

Instead

the transaction stays in its assigned arm for analysis

the fallback is flagged on the execution record

every readout reports TREATMENT FIDELITY alongside the effect:

arm	assigned	treated	fidelity
control	992,000	991,300	99.93%
variant_1	124,000	119,040	96.00% ← investigate

a fidelity gap between arms is itself a finding, and a promotion gate: an arm below its fidelity floor does not promote regardless of its measured effect

Stopping

Default run to the pre-registered sample size, then read out once. No peeking at the primary metric.

If continuous monitoring is needed, it is declared at registration and uses a sequential design with alpha spending. Ad-hoc daily checks against a fixed-sample design inflate false positives to the point where the result carries no information – and in this domain the reader is usually motivated to stop on a good day.

Guardrails are exempt: they are monitored continuously by design, because their purpose is to stop harm, not to detect an effect.

Guardrail abort

Same discipline as the migration triggers in §28: automatic, quantified, with a minimum sample so a small cell cannot fire on noise. Abort removes the cell from traffic; it does not raise a ticket.

Guardrail	Trigger
Acceptance rate	Arm below control beyond the confidence interval, sustained, at minimum volume
Unknown-outcome rate	Any elevation above the Recovery SLO (§15)
Platform-added latency	Arm exceeding the budget (§26)
Per-merchant harm	Any single merchant materially worse than control, even where the aggregate is positive
Fraud or dispute signal	Early indicator elevation, on the short-horizon proxy
Constraint violation	Any — immediate, no threshold

Aggregate improvement is not sufficient

A cell that raises approval across the portfolio while lowering it for one large merchant has not succeeded; it has redistributed. Per-merchant harm is a guardrail rather than an analysis afterthought, because the merchant experiencing the loss will discover it before we do.

Precedence against migration

A transaction in a migration ramp slice is excluded from micro cells. Running both confounds both: a movement cannot be attributed to the arm or to the new execution path, and neither readout is

usable.

ramp slice in soak → excluded from all POEM layers
slice completed soak → eligible
slice in rollback or freeze → excluded, cells terminated

Migration exposure wins. Experiments can wait; a cutover in progress cannot be paused for statistical convenience.

What is never experimented on

- Constraint enforcement — compliance is not an arm.
- Screening (\$09).
- Emergency propagation changes (\$06).
- Anything altering the amount a cardholder is charged, or the currency they are charged in.
- Disclosure and consent flows, including DCC presentation (\$20).

Disclosure

Disclosure means queryable and opt-out-able, not per-experiment notification. A merchant may ask which cells their traffic is in, may opt out entirely, and receives that opt-out at assignment rather than as a filter at analysis. Routine optimisation inside a posture the merchant has contracted for does not generate a notification for each cell; anything that changes the amount charged, the currency, or a disclosure flow is never an experiment at all (\$18). The cost of opt-out is stated honestly in \$31 item 12: if the largest merchants opt out, learning happens on the tail, where our evidence is thinnest.

Money movement

Deliberately deeper than the reviewed design, which allocates one table row each to clearing, ledger and pricing. For a bank-operated PSP this is where the regulatory weight and much of the differentiation sits — and it is where the Obligation model earns its cost.

Lifecycle after authorization



Ledger

Double-entry, event-sourced from the journal, and the authority for what is owed to whom at any moment. Every state transition that has monetary meaning produces balanced entries.

Account types

merchant_receivable	funds owed to merchant
merchant_reserve	held against future liability
scheme_receivable	funds due from network
interchange_expense · scheme_fee_expense · processing_revenue	
fbo_clearing	funds held for benefit of merchants
dispute_liability · refund_liability · chargeback_reserve	
platform_payable	platform/PayFac revenue share

Entry rules

- every entry references { event_id, obligation_id?, attempt_id? }
- ledger is derived from the journal, never written to directly
- reconciliation runs against network settlement files daily
- breaks produce cases, not alerts

Financial invariants

Daily totals agreeing to zero is necessary and not sufficient. A set of standing invariants is evaluated continuously; a breach opens a case and halts dependent processes rather than raising an alert.

Sequence invariants

- $\sum \text{authorized} \geq \sum \text{captured} \geq \sum \text{presented} \geq \sum \text{settled}$
- $\sum \text{refunded} \leq \sum \text{settled}$
- every capture traces to an authorization within its window

Balance invariants

- opening + credits - debits = closing, per account, per currency, per legal entity, per day
- network settlement = ledger settlement = merchant payable + fees + reserves + adjustments
- no cross-currency netting inside an entity position (§20)

Completeness invariants

- every monetary event has a journal event; every journal event with monetary meaning has balanced ledger entries
- no orphan authorization at clearing beyond the resolution SLO (§15)

Every monetary event carries

- event_id · economic_date · processing_date · value_date
- currency · legal_entity · account · source · destination
- amount · exchange_rate · source_event_ref

Three dates, not one: the date the economics belong to, the date we processed it, and the date value moves. Collapsing them is the most common cause of reconciliation breaks that cannot be explained after the fact.

Disputes

Dispute
└─ received · classified · reason_code · network
└─ liability who bears it, and why (authentication
 evidence from §11 determines this)
└─ evidence_required · evidence_collected · evidence_submitted
└─ representment → pre-arbitration → arbitration
└─ outcome WON | LOST | ACCEPTED | EXPIRED
└─ financial_adjustment · reserve_impact
└─ linked attempt_id · intent_id · obligation_id

Dispute intelligence feeds the decision layer
dispute probability by routing choice, credential type,
authentication outcome and merchant cohort becomes a decision
feature and an objective term (§16), closing the loop from
economic outcome back to the decision that caused it.

This is the difference between a dispute system that processes
chargebacks and one that reduces them.

Funding and reserves

Concern	Treatment
Payout timing	Funding policy per merchant; T+n or accelerated against risk assessment
Reserves	Rolling, fixed or dynamic; held in ledger, visible to merchant with reason
FBO structures	Required for platform and PayFac models; obligations discharge within the FBO before payout
Negative balance	Explicit state with a collection path, not an accounting anomaly
Split settlement	Native via obligation discharge, not post-hoc transfers
Cross-border and FX	Multi-currency positions, rate locks and conversion as explicit ledger events — specified in §20

Pricing

Cost of acceptance is one of the three things merchants actually buy, so pricing must be computable at decision time, not only at invoice time.
Routing's cost objective consumes the same tables that billing does.

Pricing inputs
 interchange (network tables, versioned, bitemporal)
 scheme fees
 processor cost
 our margin model (interchange-plus, blended, tiered)

Requirements
 routing consumes the same versioned cost tables as billing
 a merchant statement must be reconcilable to per-attempt decisions
 interchange qualification effects of message enrichment are
 attributed to the enrichment module so its value is measurable

Cross-border, FX and multi-currency

Previously an open question, and previously wrong. Treating currency as a routing attribute understates a domain that is central to the value proposition of the competitor we are most often measured against, and that carries its own regulatory surface.

The four decisions hidden inside "currency"

Decision	Made where	Consequence
Presentment currency — what the cardholder is charged in	Merchant, at intent creation	Conversion economics, disclosure obligations
Processing currency — what the authorization is submitted in	Routing, constrained	Acquirer eligibility, scheme cross-border fees
Settlement currency — what we receive from the network in	Acquiring relationship	Exposure window, conversion cost
Payout currency — what the merchant receives	Funding configuration	Merchant FX cost, local account requirements

A design that models one currency field collapses four independent decisions and makes the resulting economics unattributable. Each is a distinct field on the resource, each has a distinct owner, and the gaps between them are where FX exposure lives.

Rate locks bind to the intent

A rate quoted to a cardholder and a rate applied to an attempt must be the same rate, or the cardholder sees a different amount than they agreed to. Because an intent can produce several attempts — a step-up, a substitution, a scheduled retry — the lock cannot live on the attempt.

```
FXRateLock
├─ lock_id · intent_id
├─ pair · rate · margin_bps · inverse
├─ source_feed · feed_version · quoted_at
├─ valid_from / valid_to      (real-world validity)
├─ recorded_at                (when we captured it — bitemporal,
                             same discipline as §06)
├─ binding { cardholder_disclosed: bool, merchant_committed: bool }
├─ on_expiry  REQUOTE_REQUIRED | AUTO_EXTEND_AT_RISK | FAIL

Rules
every attempt on an intent uses the intent's lock
a retry after lock expiry requires a requote and, where the
cardholder was shown a rate, a fresh disclosure
the realised rate and the locked rate are both recorded; the
difference is booked as FX result, not absorbed silently
a rate lock is a liability from the moment it is disclosed —
it appears in the exposure position (below), not only at
settlement
```

Dynamic currency conversion

DCC is a regulated disclosure flow, not a pricing switch. It is modelled explicitly because doing it badly is a compliance finding and doing it invisibly is a merchant trust problem.

```
DCC decision
eligibility      issuer country ≠ merchant country, card
                  currency known, scheme DCC rules permit
offer           cardholder must be offered the choice, in
                  both currencies, with the margin disclosed
choice          recorded with timestamp and presentation
                  evidence — retained for the dispute window
no-choice case  if evidence of a free choice is absent,
                  DCC is not applied

Never permitted
defaulting the cardholder into DCC
applying DCC where the cardholder declined
routing selection influenced by DCC revenue — DCC margin is
excluded from the routing objective function (§12) so the
two decisions cannot contaminate each other
```

Local acquiring versus cross-border

Routing gains a currency and jurisdiction dimension:

local acquiring	higher approval rates, domestic interchange, requires local licence or partner, local settlement account
cross-border acquiring	simpler operationally, cross-border scheme fees, materially lower approval in several markets

The choice is a constraint-plus-optimization decision:

CONSTRAINT	licensing, data residency, scheme registration, merchant's legal entity structure
OPTIMIZATION	approval rate uplift versus fee delta versus settlement lag cost – learned within permitted set

Each merchant's enabled acquiring footprint is entitlement-plane state, not routing configuration. Routing chooses among what entitlement already permits.

Multi-currency ledger

The ledger holds positions per currency and does not net across them. Converting for reporting convenience destroys the exposure information that makes the position manageable.

Per currency, per legal entity	
receivable_from_scheme	settled_balance
merchant_payable	reserve_held
fbo_balance	in-flight_authorized
fx_exposure_open	hedge_position
fx_result_realised	fx_result_unrealised

Invariants

no cross-currency netting inside an entity's position
every conversion is a ledger event with { rate, source, executed_at, counterparty } – never an implicit revaluation
unrealised result is revalued on a defined schedule with the revaluation rate recorded, so the P&L is reproducible
the exposure position includes disclosed-but-unsettled rate locks, not only settled balances

Hedging

The platform holds and reports the position. Hedging policy is set by treasury, executed against the position, and recorded as ledger events. The platform does not decide hedging strategy – see §31, this boundary is not fully settled.

Payout mapping

Merchant funding configuration
per (settlement currency → payout currency → account) mapping
conversion timing: AT_SETTLEMENT | AT_PAYOUT | MERCHANT_ELECTED
who bears the margin, disclosed in the pricing plan
local account requirements per jurisdiction

Settlement lag is a first-class attribute, not an operational detail: a merchant settling T+2 in a currency they payout weekly carries an exposure window we must be able to state to them, and that window is what they are actually buying when they choose local settlement.

State machines

The internal vocabulary is large because operations needs precision. The external vocabulary is small because merchants need stability. Never export the internal set.

Payment intent lifecycle — external

requires_action · processing · authorized · partially_captured
captured · failed · canceled · partially_refunded · refunded · disputed

Internal states that must not surface

AUTHORIZATION_DISPATCHED · AUTHORIZATION_UNKNOWN
PROCESSOR_TIMEOUT · NETWORK_TIMEOUT · STAND_IN_APPROVED
REVERSAL_PENDING · REVERSAL_CONFIRMED · REVERSAL_FAILED
STEP_UP_PENDING · RECOVERY_HELD · SUPERVISOR_CASE_OPEN
CLEARING_MISMATCH · ORPHAN_AUTHORIZATION
COMPENSATION_IN_PROGRESS · LEDGER_BREAK

Order lifecycle



Journal and determinism

Current state plus complete event history. State is a projection; the journal is the record. Every meaningful transition is an immutable event, and the event schema is versioned and governed as a public contract.

```
ObligationCreated · ObligationDischarged · ObligationWrittenOff
OrderCreated · OrderCollectionProgressed · OrderCompleted
IntentCreated · IntentConfirmed
ContextAssembled · PlanResolved
ScreeningCompleted · ConstraintsBound
RiskEvaluated · AuthenticationDecided · AuthenticationCompleted
CredentialSelected · MessageEnriched · RouteSelected
AttemptDispatched · AttemptOutcomeReceived · AttemptOutcomeUnknown
StrategyDecided
Authorized · Captured · Voided · Reversed · Refunded
RecoveryOpened · RecoveryResolved · CompensationExecuted
Cleared · Settled · Funded · DisputeOpened · DisputeResolved
```

Determinism snapshot

Bound at plan resolution and stored on the request. This is what makes the question "why did this behave differently yesterday" answerable rather than a research project.

```
{
  "exec_id": "...", "plan_version": 42,
  "modules": { "screening":"4.1","constraint":"12.1","risk":"19.4",
    "authentication":"31.0","credential":"8.3",
    "routing":"23.1","strategy":"9.2","adapter":"14.5" },
  "artefacts": { "policy_set":"v311","routing_table":"v184",
    "bin_snapshot":"v52","cost_table":"v77",
    "risk_model":"fraud_v19","route_model":"route_auth_v19" },
  "bitemporal_coordinates": {
    "transaction_time": "2026-09-07T14:03:22.118Z",
    "control_plane_as_of": "2026-09-07T13:58:00Z"
  },
  "poem_assignment": { ... }
}
```

Replay and diff

```
Replay modes
SHADOW      re-execute against candidate versions, no dispatch
DIFF        original vs candidate decisions, per node
COUNTERFACTUAL estimate outcome under candidate policy using OPE
              - explicitly an estimate with a confidence interval,
              never presented as a fact

Guardrails
replay may never dispatch to a processor
replay of a transaction containing credentials runs inside the
  PCI boundary with the same masking rules as production
every replay is itself logged with the operator identity
```

Correction to the reviewed design

The reviewed document's replay section presents a candidate route's expected approval rate as a bare number. Producing that number credibly is the entire difficulty: it is an off-policy estimate, it requires logged propensities, and it has a confidence interval that is often wide enough to change the conclusion. Presenting it without the interval would give operations false confidence in exactly the decisions where care matters most.

Error model

Processor codes never reach merchants. A canonical classification is produced for every failure, and it carries the fields that determine what anyone should do next.

```
CanonicalResult
├── code          CARD_DECLINED · INSUFFICIENT_FUNDS · INVALID_CARD
│               · EXPIRED_CARD · AUTHENTICATION_REQUIRED
│               · AUTHENTICATION_FAILED · POLICY_BLOCKED
│               · SCREENING_BLOCKED · RISK_BLOCKED
│               · PROCESSOR_UNAVAILABLE · NETWORK_TIMEOUT
│               · DUPLICATE · INVALID_REQUEST · UNKNOWN
├── class        SOFT | HARD | TECHNICAL | UNKNOWN
├── retryability { retryable: bool, actions_permitted[],
│               earliest_retry_at }
├── customer_action what the cardholder must do, if anything
├── merchant_action what the merchant should do, if anything
├── liability      who bears loss if this proceeds
└── raw (restricted, §24)
    processor_code · network_code · iso_field · processor_message
```

The `class` field is what Attempt Strategy consumes; `customer_action` and `merchant_action` are what merchant support consumes. Separating them means a decline taxonomy change does not require a support retraining, and vice versa.

Observability and disclosure

Four tiers of visibility, and — a point the reviewed design misses — disclosure is not a static table because delegation changes it. A PayFac's access to submerchant data is a policy question answered by the control plane, not a column in a spreadsheet.

Data	Merchant	Platform / PayFac	Operations	Engineering
Status, amount, last four, network	Full	Full	Full	Full
Canonical decline class and actions	Full	Full	Full	Full
Authentication result and liability	Full	Full	Full	Full
Attempt sequence and strategy origin	Summary	Full	Full	Full
Processor and acquirer identity	Contractual	Contractual	Full	Full
Route decision and reason	Reason only	Reason only	Full	Full
Risk score	Band	Band	Full	Full
Constraint decision and authority level	Reason only	Reason + own rules	Full	Full
Exec ID and plan version	Reference only	Reference only	Full	Full
Module versions and dependencies	No	No	Full	Full
Experiment assignment	On request	On request	Full	Full
Raw network message	No	No	Restricted	Restricted
Credentials	Never	Never	Never	Never

Delegation overrides the table

A submerchant's data is visible to its PayFac only where the delegation grant and the applicable privacy regime permit it, and a platform's access to its merchants' data is bounded by the same mechanism. A merchant operating under a platform may also have contracted for visibility the platform does not want them to have. These are constraint-plane questions, resolved per request, and logged — not a static access matrix.

Rejected alternatives — the "why not" view

Every decision already logs its considered action set and the reason it chose within it. Exposing the rejected options costs nothing to produce and is disproportionately valuable to sophisticated merchants, who generally want to know why the obvious option was not taken.

Selected processor_a
 +48 bps predicted approval · lower cost · healthy

Rejected processor_b BIN not supported for this acquiring entity
 processor_c higher expected dispute cost for cohort
 processor_d health degraded at decision time
 processor_e would breach concentration bound (§16)

Available at the integrator and operations tiers by default, and at the merchant tier by entitlement. Reason codes are canonical, not free text, so they are queryable and countable — "how often was processor_b excluded for acquiring entity reasons last month" is a supportable question.

Payment intelligence as a product

The observability model is a differentiator, not an internal tool. Merchants see the outcome by default and can ask why: authentication decision, credential choice, routing rationale, retry logic, experiment participation. What-if queries — the expected effect of requiring authentication above a threshold, or moving a cohort to network tokens — are answered from the same replay and off-policy machinery that serves internal evaluation (§22), with confidence intervals and an explicit "insufficient evidence" answer where the logged data does not support the counterfactual.

The limit on what-if

A what-if answer is an off-policy estimate and inherits every constraint from §16: it requires overlap between the logged behaviour and the proposed alternative. Where a merchant has never had traffic on the alternative, the honest answer is that we cannot tell them, and the product must be able to say so rather than returning a confident number derived from a different population.

Experiment disclosure

A merchant is entitled to know whether their traffic is in an experiment and to opt out. This is uncomfortable commercially and non-negotiable for enterprise accounts; hiding it is how a platform loses a strategic client permanently.

API surface

Two tiers, one model

The surface is tiered so that the common case is one call and the complex case is possible. Both tiers produce the same objects; the first simply constructs the upper layers for you.

```
TIER 1 – the path almost everyone takes
POST /v1/payment_intents          obligation and order implicit
POST /v1/payment_intents/{id}/capture
POST /v1/payment_intents/{id}/refund
POST /v1/payment_methods
Idempotency-Key on every mutating call

TIER 2 – reached only when the commercial structure needs it
POST /v1/obligations              multi-party, scheduled, or
                                   surviving the collection
POST /v1/orders                  multi-payer or multi-instrument
POST /v1/payment_requests         explicit control of individual
                                   instructions against an intent

Rule for the API team: no field required in tier 1 that a standard
card checkout does not already have. If a new capability needs a
new required field at tier 1, it belongs at tier 2 with a default.
```

Decision contract

Internal, and the reason new decision domains do not require orchestrator changes. Every decision point speaks one language.

```
POST /internal/v1/decisions/evaluate
{ "decision_point": "ROUTING", "context_ref": "...",
  "candidate_actions": [...] }

→ { "decision": "...", "strategy_id": "...", "strategy_version": "...",
    "experiment": {...}, "regime": "LEARNED",
    "expected_value": {...}, "propensity": 0.72,
    "considered": [...], "rejected": [{action, reason_code}],
    "fallback_would_have_been": "...", "reason_codes": [...] }

MANDATORY, enforced by schema validation at registration
propensity                probability this action was chosen.
                           1.0 for deterministic strategies –
                           populate it anyway, so no consumer
                           needs a special case
fallback_would_have_been  what the declared fallback strategy
                           would have chosen
rejected[]                with canonical reason codes

A decision point that cannot populate all three fails validation
and cannot be registered. Neither propensity nor the fallback can
be backfilled for traffic that has already run.

Computing the fallback: evaluate inline where it is cheap and
deterministic; otherwise record the decision inputs and recompute
it offline by replay (§22). Replay already exists, the inputs are
already logged, and this keeps a second model call off the
authorization path.
```

Full surface

```
POST /v1/obligations
GET /v1/obligations/{id}
POST /v1/obligations/{id}/discharge

POST /v1/orders
GET /v1/orders/{id}
POST /v1/orders/{id}/cancel

POST /v1/payments              alias: create + confirm in
                                one call, the default path
                                when confirmation is separate
POST /v1/payment_intents
GET /v1/payment_intents/{id}
POST /v1/payment_intents/{id}/confirm
POST /v1/payment_intents/{id}/capture
POST /v1/payment_intents/{id}/cancel
POST /v1/payment_intents/{id}/refund
```

```

GET    /v1/payment_intents/{id}/attempts
GET    /v1/payment_intents/{id}/events

POST   /v1/payment_methods          GET /v1/payment_methods/{id}
POST   /v1/customers             GET /v1/customers/{id}
POST   /v1/authentication_sessions GET /v1/authentication_sessions/{id}
POST   /v1/credentials           GET /v1/credentials/{id}
POST   /v1/credentials/{id}/forward ← Forward API, §08
POST   /v1/transfers             GET /v1/transfers/{id}
GET     /v1/settlements/{id}      GET /v1/payouts/{id}
GET     /v1/disputes/{id}

```

Webhook and event delivery

The event journal is internal; webhooks are the merchant's view of it, and their delivery semantics are part of the public contract. Getting this wrong is the most common source of merchant-side duplicate fulfilment.

```

Guarantees
  at-least-once delivery    duplicates are possible by design
  event_id                 stable, idempotent for the consumer
  sequence                 per payment intent, monotonic;
                           global ordering is not offered
  signature                every payload signed; key rotation
                           published in advance

Retry
  exponential backoff to a bounded horizon, then dead-letter
  delivery state is queryable per event (§24 tiering)
  replay of a delivered or failed event on request, with the
  original payload – never a regenerated one, which would
  reflect state that has since moved

What we do not do
  guarantee ordering across payment intents
  guarantee exactly-once delivery (§00 non-goals) – the consumer
  contract is idempotent handling, stated in the documentation
  rather than discovered in production
  hide event schema versions: payload versions are pinned per
  endpoint and migrated on the merchant's schedule

Sandbox
  every canonical result (§23), including the awkward ones –
  timeout with unknown outcome, late approval after abandonment,
  step-up mid-flow, partial capture – is producible on demand.
  A sandbox that only produces approvals and declines teaches
  developers to write integrations that break on the third case.

```

Internal operations

```

GET    /internal/v1/payment_intents/{id}/timeline
GET    /internal/v1/payment_intents/{id}/execution
GET    /internal/v1/attempts/{id}/trace
GET    /internal/v1/attempts/{id}/decisions
GET    /internal/v1/attempts/{id}/message          (restricted)
GET    /internal/v1/attempts/{id}/determinism
GET    /internal/v1/recovery/cases
GET    /internal/v1/recovery/cases/{id}
GET    /internal/v1/constraints/resolve            (what applies, and why)
GET    /internal/v1/plans/{exec_id}/{version}

POST   /internal/v1/attempts/{id}/replay          (shadow only)
POST   /internal/v1/payment_intents/{id}/retry
POST   /internal/v1/payment_intents/{id}/reverse
POST   /internal/v1/recovery/cases/{id}/resolve

All internal endpoints: RBAC, dual control on mutating operations,
full audit, credential masking enforced at serialisation.

```

Adopted from the review

The decomposed internal surface is directly influenced by the reviewed design's §44, which is the strongest part of that document. The changes: endpoints hang off attempts rather than payments where the data is attempt-scoped, replay is shadow-only by construction, and `/constraints/resolve` is added because "why did this rule apply" is the most common operations question and it had no endpoint.

Latency and data architecture

The reviewed design claims under 40 ms of added latency in one section and under 300 ms in another, with no screening on the path and a synchronous risk call. Neither number is defended. Below is a budget that includes everything a bank must do, stated with the assumptions that make it achievable.

Stage	p50	p99	Assumption that makes it achievable
Edge — authn, idempotency, schema	3 ms	9 ms	Token validation cached; idempotency in local store
Resource + context assembly	2 ms	6 ms	Instrument attributes arrive with credential (\$08)
Plan resolution	0.5 ms	2 ms	Lookup against pushed plans; no rule evaluation
Screening (parallel)	5 ms	12 ms	In-memory list, refreshed out of band
Constraint bind (parallel)	1 ms	3 ms	Constraint set precompiled
Credential select (parallel)	1 ms	3 ms	No remote call
Risk	8 ms	22 ms	Co-located inference, features precomputed
Authentication decision	1 ms	3 ms	Decision only; protocol is a separate flow
Routing	1.5 ms	4 ms	Tables in memory, model co-located
Message enrichment	1 ms	3 ms	All inputs already in context
Adapter serialisation	1 ms	3 ms	—
Orchestration overhead	2 ms	6 ms	DAG scheduling, record emission is async
Platform-added, parallelism applied	~22 ms	~58 ms	Critical path, not sum of stages
Processor + network + issuer	180 ms	900 ms	Outside our control; dominates

The honest statement of the target is **platform-added latency under 60 ms at p99**, with screening included and risk on the path. The end-to-end number is dominated by the issuer and is not a platform commitment. Where 3DS challenge is invoked, end-to-end latency is a user-interaction duration and belongs to a different measurement entirely — conflating the two is how competitors publish flattering numbers.

Latency budget ownership

Each decision point owns a hard budget. A module over budget is a product regression raised like any other defect, not an observability curiosity — the same discipline the operating model applies to explanation budgets.

Path or module	p99 budget	Owner
Simple card authorization, end to end platform-added	50 ms	Platform
Card with risk and authentication decision	60 ms	Platform
Screening	12 ms	Screening
Risk	22 ms	Risk
Credential selection	3 ms	Credential
Routing	4 ms	Routing
Constraint bind	3 ms	Control plane
Authentication decision	3 ms	Authentication
Message formatting	3 ms	Message intelligence
Orchestration overhead	6 ms	Platform
3DS challenge, recovery, replay	Excluded	— user interaction or asynchronous

Data placement

H0T — in process, or one hop, never a query

plan bundles · constraint sets · routing tables · cost tables
merchant configuration · sanctions lists · model artefacts
instrument attributes (carried with the credential reference)
Refresh: push, versioned, signed, with rollback

WARM – asynchronous, off the authorization path
event journal (append-only, partitioned)
transactional store for resource state (distributed SQL)
decision records for learning
operations search index

COLD – reporting, training, reconciliation
lakehouse · settlement files · model training sets · archives

Rule: nothing on the authorization path issues a query whose latency
we do not control. If a module needs data it does not have, the
answer is a push channel, not a lookup.

Retention and residency

credentials	per PCI scope, in vault only, tokenised elsewhere
authentication evidence	per scheme requirement (liability window)
raw network messages	short, restricted, encrypted at rest
decision records	long – required for OPE and model validation
journal	long – required for dispute and audit
personal data	per jurisdiction; residency zone is a context attribute and constrains where a plan may execute

Resilience and deployment

Every module declares its own degraded behaviour. The orchestrator enforces deadlines and applies the declared disposition; it never improvises a fallback, because an improvised fallback is business logic in the orchestrator.

Module	Disposition	Degraded behaviour
Screening	HARD_FAIL	None. Transaction fails.
Constraint bind	HARD_FAIL	None; last-known-good set is used only if signed and unexpired
Risk	FALLBACK	Per merchant policy: approve, require authentication, or decline
Authentication decision	FALLBACK	Conservative default from constraint set
Credential select	FALLBACK	Default credential per preference
Routing	FALLBACK	Static route table, health-filtered
Message enrich	DEGRADE	Proceed with base message; enrichment absence recorded
Authorize	RECOVER	Hand to Recovery Supervisor (§15)

Parallel stack traffic planning

Traffic is planned, not percentaged. A plan version's exposure is defined as a predicate over context, so exposure is explainable:

```
plan v43 exposure:
  include  country=US AND channel=ecommerce AND amount<10000
  exclude merchant IN strategic_accounts
  exclude merchant.autonomy_preference = MANUAL
  sample   1% of remainder, stable hash on merchant+customer
```

Stable hashing means a given customer sees consistent behaviour across their session and across retries. Random per-transaction assignment produces incoherent merchant experiences and invalidates attempt-level analysis.

Rollback is a plan-version pointer change: seconds, no deployment.

Tenant isolation

A reliability property, not a capacity setting. One platform client can represent thousands of submerchants, so a single tenant's spike must not be able to consume another's capacity anywhere in the system — including the paths nobody thinks of as shared.

Quotas enforced per tenant, at every tier

```
request concurrency · request rate
compute and memory share
event stream partition share
queue depth for asynchronous work
recovery case capacity          ← the one usually missed:
                                one tenant's incident must not
                                starve another's recovery

experiment assignment capacity
replay and analytics capacity
```

Breach behaviour is shed, not queue: exceeding a quota returns a canonical rate-limit result immediately rather than consuming capacity while waiting. A queue is a slower way to fail that also degrades everyone else.

Failure domains

Region isolation	a region processes with locally-pushed artefacts and does not depend on a control-plane round trip
Processor isolation	one processor's failure is a routing input, not an outage
Merchant isolation	per-client concurrency limits and load shedding so one merchant's spike does not starve others
Blast radius	a plan version's exposure predicate bounds the population any single change can affect

Migration and legacy coexistence

Previously the highest-risk unresolved item in the specification. It remains the highest-risk item in the programme, but it is now a design rather than an acknowledged gap.

The failure this section exists to prevent

A facade over a legacy engine, plus a new journal, plus both systems able to mutate the same payment, produces split-brain: two systems that each believe they hold the truth about a transaction, diverging silently, discovered at settlement. Every mechanism below exists to make that state unreachable rather than unlikely.

Single writer per resource class

At any moment, exactly one system may mutate a given resource class for a given traffic slice. Ownership is explicit, recorded on the resource, and changes only by deliberate cutover.

Resource	Phase 1	Phase 2	Phase 3
credentials	v3	v3	v3
payment_intents	legacy	v3	v3
attempts	legacy	legacy	v3
clearing/settlement	legacy	legacy	v3
ledger	legacy	legacy	v3

Every resource carries:
{ owning_system, owned_since, cutover_id }

A write to a resource this system does not own is rejected – not queued, not merged, not reconciled later. The rejection is the safety property; anything softer reintroduces divergence.

Dual-write is not adopted

Writing to both systems and reconciling afterwards was proposed and is rejected. Dual-write has no atomicity across two stores, so its failure mode is precisely the divergence the pattern is meant to prevent — and the divergence appears under load, when reconciliation capacity is lowest. Single ownership with one-way propagation is slower to migrate and does not have that failure mode.

One-way propagation

LEGACY-OWNED RESOURCES

legacy engine

→ transactional outbox (same transaction as the state change, so no lost or phantom events)

↓ change data capture

↓ translation layer legacy shape → v3 event schema

↓ v3 EVENT JOURNAL (§22)

→ v3 read models, reporting, learning

→ reconciliation

v3 reads are eventually consistent with legacy writes and are labelled as such. v3 does not mutate legacy-owned resources.

V3-OWNED RESOURCES

v3 journal → projection → legacy read views, for as long as legacy reporting and operations tooling need them

Cutover

Per pipeline, per traffic slice, using the same exposure predicate mechanism as plan versions (§27) – so migration exposure and release exposure share one control surface and one rollback path.

1 SHADOW v3 executes alongside legacy, dispatches nothing,

- | | | |
|---|---------|---|
| 2 | MIRROR | decisions diffed against legacy outcomes
v3 owns reads; legacy still owns writes; divergence
measured continuously against reconciliation
invariants |
| 3 | CUTOVER | ownership marker flips for the slice; legacy
becomes a projection consumer |
| 4 | DRAIN | in-flight legacy-owned transactions complete under
legacy ownership – ownership never changes for a
transaction already in flight |
| 5 | RETIRE | legacy path disabled for the slice |

Rollback is an ownership-marker flip plus a drain, available at every phase up to RETIRE.

Reconciliation invariants

Continuously evaluated during coexistence. A breach halts cutover progression automatically; it does not raise a ticket for someone to triage.

- Every legacy authorization appears in the journal within the propagation SLO, or an exception case is opened.
- Monetary totals per merchant per day agree between systems to zero, not to a tolerance.
- No resource shows mutations from two systems within one ownership epoch.
- Every journal event traces to either a v3 execution or a captured legacy mutation, with no orphans in either direction.
- Ownership markers are consistent across regions.

What "shadow agreement" can and cannot mean

A gate that would never open

Requiring zero decision discrepancies between shadow and legacy before advancing is a gate that never opens, because the new engine is *supposed* to decide differently — better routing, step-up recovery, suppressed optimization on stale attributes are all intended divergences. Holding out for zero either stalls the migration indefinitely or, worse, gets satisfied by tuning the new engine to imitate the old one, which discards the entire reason for building it.

Two classes of comparison, two different bars.

INVARIANTS – bar is zero, always
monetary totals per merchant per day
event completeness: every legacy authorization in the journal
no resource mutated by two systems in one ownership epoch
ownership marker consistency across regions
vault and network-token allocation reconciliation

DECISIONS – bar is explained, not identical
every divergence attributable to a known, intended change,
by category and expected direction
unexplained divergence rate below threshold and trending down
no divergence category that nobody can account for

The gate is: invariants at zero, and the unexplained decision
divergence bucket empty. Explained divergence is the product
working.

Ramp and gates

Exposure remains predicate-based (\$27) rather than a bare percentage, because "5% of traffic" is not a population anyone can reason about afterwards. The ladder below is the sequence in which predicates are widened.

Step	Population	Minimum soak	Advance requires
0	Synthetic and internal test accounts	Continuous	Invariants at zero
1	JPM-as-merchant traffic	7 days	Invariants at zero; unexplained divergence empty
2	Opt-in mid-market, low ticket, single currency, domestic	14 days	Above, plus acceptance rate non-inferior
3	Opt-in mid-market, full ticket range	14 days	Above, plus platform-added latency within budget
4	General mid-market, non-opt-in	30 days	Above, plus a full settlement cycle reconciled
5	Enterprise, by named account, with the account informed	Per account	Above, plus account-level agreement

Cross-border, multi-currency and card-present slices are ramped as separate ladders rather than as steps within this one; they exercise different code paths and their failure modes are unrelated. Strategic accounts are never in an early step, and never in the same step as the first exercise of a

capability.

Rollback triggers

Automated, evaluated continuously, and expressed against the metric that actually matters rather than a convenient proxy. Each trigger carries a minimum sample size so that a narrow slice cannot fire it on noise.

Trigger	Threshold	Action
Acceptance rate regression	Approval rate below matched legacy baseline beyond the confidence interval, sustained across two intervals at minimum volume	Halt ramp, then revert slice
Write-path error rate	5xx above 0.05% of slice volume over a rolling 3 minutes, minimum 2,000 transactions	Immediate revert of slice
Platform-added latency	p99 platform-added latency above budget by more than 20 ms for two consecutive 5-minute intervals	Halt ramp; revert if sustained
Unknown-outcome rate	Unresolved-outcome age or rate above the Recovery SLO (\$15)	Immediate revert of slice
Ledger or reconciliation divergence	Any non-zero invariant breach	Freeze ownership, drain, supervisor case — see below
Emergency constraint propagation failure	Any node failing to confirm within deadline (\$06)	Slice stops accepting traffic, fail-closed

Two corrections to the proposed thresholds

A 150 ms p99 latency tolerance was proposed. That is two and a half times the platform's entire added-latency budget (\$26), and measuring it end-to-end conflates our contribution with issuer response time, which we do not control. The trigger is stated against platform-added p99 instead, at a tolerance the budget can actually detect.

Immediate revert on ledger divergence was also proposed. Reverting while a monetary inconsistency is open moves in-flight transactions across an ownership boundary during the exact window in which state is known to be wrong, which can deepen the break. The correct action is to freeze ownership, drain in flight, open a supervisor case, and revert only once the position is understood.

Rollback mechanics and the point of no return

Rollback is an ownership-marker flip plus a drain (\$28, cutover). It is never a data migration, and never a merge of divergent state.

in-flight transactions	complete under current ownership
new transactions	resolve to the reverted owner
journal entries	retained; a reverted slice keeps its history, which is what makes a second attempt cheaper than the first

POINT OF NO RETURN – stated explicitly, per resource class

Rollback is available until the new system has written state the legacy system cannot represent. For each resource class, that boundary is named before cutover begins:

payment_intents	first intent with multiple obligations
attempts	first step-up-recovered attempt
ledger	first multi-currency position (\$20)
obligations	first obligation outliving its order

After the boundary, forward-fix is the only path. A migration plan that does not name this boundary has one anyway – it simply discovers it during an incident.

What ships first

Ordered by risk-adjusted value, which is not the same as ordered by architectural elegance. The resource model and the vault carry real value before the execution engine exists, and both can be delivered without taking write ownership of a live payment.

- | | | |
|----|--------------------------------------|---|
| 1 | Credential vault and Forward API | new writes only, no migration |
| 2 | Resource model as a facade | read-through, legacy owns state |
| 3 | PIH enrichment and BIN intelligence | feeds legacy routing first |
| 4 | Journal ingestion via CDC | observability before control |
| 4a | Decision logging on the legacy stack | propensity, considered set and fallback emitted from current routing and retry. |
| | | No behaviour change; builds |

	the evaluable history that
	OPE will need (\$16)
5 Control plane in advisory mode	decisions computed, not applied
6 Execution engine, shadow	diffed against legacy
7 Cutover by slice	narrowest first

Commerce device platform

Card-present has been listed as assumed rather than specified since v3.0. For QSR, fuel, retail, lodging and transit — segments where our existing merchant relationships are strongest — the terminal is not an API client, and treating it as one understates a platform of its own.

Why a terminal is not a client

An API client is stateless, updatable at will, and observable.
A terminal estate is none of those:

it holds cryptographic keys that must be injected securely

it runs firmware and an application with independent lifecycles

it must transact while disconnected

it is physically accessible to attackers

it is replaced on a hardware cycle measured in years

a bad configuration push bricks revenue at a physical location

Domain scope

Device identity	per-device certificate, attestation, lifecycle
Key injection	remote key loading, P2PE, PIN key hierarchy — anchored in the HSM boundary (§10)
Estate management	inventory, location, merchant mapping, ownership, RMA
Firmware and app	versioned, staged, rollback-capable — the same exposure discipline as plan versions (§27), applied to physical estate
Configuration	terminal config as a control-plane artefact, not a per-device setting (§06)
EMV kernel	contact, contactless, kernel versions, certification state per acquirer and network
Offline operation	floor limits, offline approval, deferred submission, and the risk position that creates
Telemetry	device health, decline patterns by device, tamper events

Integration path

Terminal	
↓ device identity, attested	
Device gateway	persistent connection, store-and-forward
↓	
Payment edge (§02)	same idempotency and authn discipline
↓	
Transaction context (§05)	gains device, location, entry mode, PIN capability, offline indicator
↓	
Decision and execution	unchanged — a card-present transaction is a different context, not a different platform

The decision that gates this

Whether we build a device platform, partner for one, or restrict card-present to existing estate integration is a strategic choice with a multi-year lead time and a hardware certification calendar that does not compress. It is not resolved here. What is resolved is that it cannot be handled as a set of fields on the transaction context, which is how v3.0 through v3.4 implicitly treated it.

Vertical extensions

Verticals attach context; they never add fields to core objects. The moment a lodging field appears on the payment object, every future vertical has a claim to do the same, and the core becomes unmaintainable.

PaymentIntent

└ industry_context_ref → typed context object

LodgingContext

reservation, check-in/out, folio, estimated total, incremental authorization schedule, no-show handling

TravelContext

ticket number, passenger, itinerary legs, agency, carrier, ancillary items

HealthcareContext

HSA/FSA eligibility, benefit type, substantiation, IIAS requirements

FuelContext

pre-authorization amount, completion amount, product codes

RestaurantContext

tip adjustment window, server, check

MarketplaceContext

seller obligations, split terms, delivery trigger

UtilityContext

account, billing period, convenience fee treatment

Each context declares which execution nodes it modifies. Lodging changes incremental authorization and reversal handling; healthcare changes credential eligibility and message content. The core is untouched in both cases.

Unresolved questions

Eighteen things this document does not answer. The pattern is consistent across revisions: cross-border and FX, legacy state ownership, and now strategy authoring each moved from an open question to a section, and each generated new open questions on the way. That is what designing something properly looks like, and a list that shrinks is more likely to mean the questions stopped being asked than that they were answered.

#	Question	Consequence if unresolved
1	Settlement and funding remain shallower than the authorization path	The domain with the most regulatory weight still has the least design
2	Dispute lifecycle is sketched, not designed	Representment and network dispute rails will be built ad hoc
3	Boundary between platform FX position and treasury hedging authority	Either treasury inherits a position it cannot control, or the platform starts making hedging decisions it should not
4	DCC as a revenue line versus DCC as a cardholder service	Commercial policy question with compliance consequences; §20 excludes it from routing but does not settle the pricing posture
5	Local acquiring footprint — build, partner or defer per market	Determines whether cross-border approval-rate parity with Adyen is achievable at all
6	Whether learned strategy can be defended to enterprise procurement	If not, autonomy levels 3+ are internal-only and the volume advantage shrinks
7	Data residency versus centralised learning	May force per-region models with materially less data each
8	Obligation's boundary against the enterprise general ledger	Risk of building a second GL by accident
9	Non-card rails: RTP, FedNow, open banking and push payments have semantics this design assumes away	Recovery (§15) and Attempt Strategy (§14) are card-shaped; the Rails domain exists but the design does not
10	Terminal and card-present estate integration	Context assembly assumes richer terminal data than legacy estates provide
11	Organisational feasibility of 28 owned domains	Under-staffed domains become shared-service bottlenecks and recreate the monolith socially
12	Whether merchant-visible experiment opt-out materially degrades learning	If large merchants all opt out, learning happens only on the tail where we are thinnest
13	Individual rail designs — ACH returns, RTP return requests, open banking consent lifecycle	§13 makes the execution model rail-agnostic; it does not design the rails. Each needs its own pass before that rail carries volume
17	Card-present: build a device platform, partner, or restrict to existing estate	Multi-year lead time and a hardware certification calendar that does not compress. §29 scopes the domain; the strategic choice is unmade
18	Population overlap detection over arbitrary predicates (§18)	The controlled segment vocabulary makes the uniqueness check tractable and caps concurrency at the number of segments. Whether that cap binds before a general solution is found is unknown, and a general solution is expensive
14	Who owns the bounds on the approval-versus-cost exchange rate (§16)	The objective function is a commercial policy wearing a model artefact's clothing; if product, risk and finance do not agree who sets its bounds, the optimizer settles it by default
15	Strategy contention under client self-serve authoring (§17)	Disjoint populations resolve contention for internal authoring; at self-serve scale, either strategies are scoped so narrowly they are useless or attribution degrades to "something changed". Tier 3 should not be promised until this is answered
16	Who allocates the platform-funded exploration pool (§17)	Exploration is a finite resource denominated in lost approvals. Merchant-scoped budgets are solved; the platform pool has no named owner, and an unowned budget is spent by whoever asks first

The one to watch

Item 5 is the item most likely to invalidate the commercial case rather than merely delay it. Every other open question affects how well we execute this architecture; local acquiring footprint affects whether the architecture can deliver the acceptance rates the strategy assumes in markets where we do not acquire locally. It deserves a decision on a shorter timeline than the rest.

Change log and review disposition

Structural additions in 3.0, against v2.2

Change	Section	Origin
Recovery and compensation as a named domain with an owner	\$15	Gap review — unowned in both v2.2 and the reviewed design
Screening as a first-class auth-path stage with HARD_FAIL disposition	\$09	Gap review — absent from both
Attempt Strategy replaces separate Retry and Authentication step-up	\$14	Stripe pressure test, formalised here
Compensation graph declared in the execution plan	\$07, \$15	New
Adapter behaviour attributes as production gating requirement	\$12	New — dual/single message semantics leak
Money movement expanded: ledger accounts, funding, reserves, FBO	\$19	Gap review
Reg II, PINless and least-cost routing specified explicitly	\$12	Gap review — PIH investment had no routing-side specification
Latency budget restated with screening included and defended	\$26	Gap review — prior numbers were undefended
Disclosure as delegation-resolved policy, not a static matrix	\$24	Gap review
Unresolved questions stated in the document	\$31	New

Working-session corrections (3.6)

Arising from a whiteboard model of the decision and experimentation path. The sketch was a better articulation of the execution unit than this specification had, and it surfaced three failure modes that discipline would not reliably prevent.

Change	Origin	Where
Flow as the named unit of execution. Human-readable binding of a plan, with an enforced naming convention. Sequential names rejected at registration	Adopted from the working model — better than the opaque plan identifier this specification used	\$17
Single-difference variant construction. <code>diff_count = 1</code> enforced by the compiler	Implicit in the working model; made explicit here. Reconciles flow-level allocation with per-decision attribution	\$17
Allocation ledger with a controlled segment vocabulary; uniqueness on (layer, segment); three-arm shared control as the preferred resolution	The sketch had two experiments allocating the same population at the same decision point	\$18
Propensity and fallback mandatory in the decision contract, enforced by schema validation at registration	The sketch's output metadata carried experiment and ExecID but neither of these	\$25, \$17
Decision logging on the legacy stack, immediately	Neither field can be backfilled; waiting for migration discards evaluable history	\$16, \$28
<code>experimentable: false</code> on decision points; compiler rejects variants differing there	Screening was absent from the sketch's flow and must sit before assignment	\$17, \$18
Permanent holdout, 2% default, excluded from exploration as well as experiments	Not raised. Cannot be established retroactively, and cumulative platform effect is unmeasurable without it	\$18
Treatment fidelity reported alongside effect; fidelity floor as a promotion gate	Not raised. A fallback at the experimented decision point means the treatment was not delivered; dropping those cases biases toward healthy periods	\$18

Investment review disposition (3.5)

A CTO-level review raised twenty structural gaps and fifteen proposed capabilities. Thirteen are incorporated here; the full item-by-item record, including what was declined and why, is in the companion decision log rather than duplicated in this appendix.

Incorporated	Where
Platform security architecture — zones, key hierarchy, identity, replay as a security boundary	\$10, new
Rail execution model — rails declare their own shape rather than being forced into authorize-capture	\$13, new
Commerce device platform — terminal estate as a domain	\$29, new
Decision point abstraction — uniform shape for every decision	\$17
Strategy compiler — inexpressibility made mechanical rather than asserted	\$17
Exploration budget manager — permission to learn before what to try	\$17
Economic decision policy — hard constraints, economic constraints, objective	\$16
Outcome graph — strategies evaluated on economic outcome, not approval	\$16
Causal attribution, with identifiability limits stated	\$16
Policy algebra — action space as a computed set	\$06
Effective configuration snapshot	\$05
Financial invariants and three-date monetary events; dispute lifecycle feeding decisions	\$19
Latency budget ownership per decision point; tenant isolation quotas; decision contract API; rejected-alternatives view	\$26, \$27, \$25, \$24

Decision platform (3.4)

Not a review response — a scope addition. The learning layer specified how the platform learns but left the unit of change as the module version, which routes every product judgment through an engineering release. \$17 makes strategy a first-class deployable artefact over the same substrate.

Addition	Why it was missing
Strategy object with its own lifecycle, expiry and fallback	The spec had modules, models and constraints, and assumed strategy lived inside modules
Constraint-bounded authoring — non-compliant strategies inexpressible, not rejected	Every new authoring surface is a new way around the authority ladder unless the action set is derived rather than declared
Three governance classes split on fitted parameters	Uniform model governance over deterministic rules would make the fast path as slow as the slow path
Backtest → OPE → pilot → ramp as one workflow	All three existed, owned by three teams, reachable only as separate requests
Per-strategy attribution against a declared fallback	Without it, strategies accumulate and none can be retired
Contention rules and inert-strategy reporting	The layer registry permits one cell per layer; concurrent strategies collided with that and the collision was silent
Exploration budget scoped to the population bearing the cost	A shared finite budget denominated in lost approvals had no allocation rule
Authoring tiers, sequenced	Internal capability, internal speed and client self-serve are three products with different blast radii; conflating them promises the third on the first's timeline

Two new open questions came out of it: contention under self-serve authoring, and ownership of the platform-funded exploration pool (\$31, items 15 and 16).

Coverage audit (3.3)

This document was audited against the v2.2 module inventory and against the five areas the Stripe whitepaper pressure test identified. Consolidation into 3.0 dropped material that v2.2 had, which is the usual cost of a rewrite and the reason the audit was worth running.

Pressure-test area (v2.2)	State in 3.2	State in 3.3
Experimentation	Plumbing across six sections; no lifecycle, no unit of randomisation, no stopping rule, no registry	\$18, a section of its own
3DS as retry	Covered	\$14, unchanged
Dynamic authorization formatting	Lost — Message Intelligence existed as a domain with no specification	\$12, versioned per issuer, its own POEM layer
Multi-prediction fraud	Lost — a single fraud score	\$16, risk as a vector including false-decline prediction
Contextual bandit optimization	Lost — OPE was specified with no exploration mechanism to generate the logs it needs	\$16, budgeted in basis points of approval rate

Other gaps the audit found

- **Non-card rails had no owner.** v2.2 claimed coverage of all rails; 3.0–3.2 quietly became a card specification without saying so. 3.3 adds the domain, and adds an explicit statement that the design is card-shaped rather than a claim that it generalises (§03, §31 item 9).
- **Security architecture was absent entirely** — no key management, no cryptographic boundary decomposition, no service-to-service authorization, across three versions. Named as a domain now and listed as the largest open item (§03, §31 item 13). Naming it is not the same as designing it.
- **Webhook delivery semantics were undefined** despite webhooks being the merchant's primary view of the event model, and despite §00 making simplicity a binding constraint (§25).
- **Sandbox fidelity** — a sandbox producing only approvals and declines teaches developers to write integrations that break on the first unknown-outcome timeout (§25).
- **Exploration cost had no commercial owner.** Related to open question 13: exploration spends approval rate, and that cost lands on identifiable merchants. It is now expressed in basis points rather than as a traffic percentage, so someone can approve it.

Deliberately not re-added

- **The v2.2 count of 32 modules.** Domains and modules are different units — one domain owns several versioned modules — and reconciling the two counts would imply a mapping that does not exist. The domain inventory (§03) is the authority.
- **The four-phase roadmap from the v2.2 deck.** Superseded by the migration sequence in §28, which is ordered by risk-adjusted value rather than by capability grouping.

Disposition of the migration and learning-layer review (3.2)

A focused review of cutover sequencing and off-policy evaluation gates. Most of it strengthens sections that were thin; three specific proposals are changed and one is rejected outright.

Recommendation	Disposition	Where
Quantified rollback triggers on error rate, latency and ledger divergence	Adopted — the specification previously said cutover halts on breach without saying what a breach is. Two thresholds restated; acceptance-rate regression and unknown-outcome rate added	\$28
Graduated canary by merchant segment with volume caps	Adopted as a predicate ladder with soak periods and explicit advance criteria; cross-border and card-present ramp as separate ladders	\$28
Propensity distribution audits to avoid severe clipping	Adopted and promoted to a named gate with effective sample size and policy-overlap diagnostics	\$16
Poisoned reward signal stress testing	Adopted and extended to counterparty influence and feedback-loop detection, with automatic autonomy downgrade rather than an alert	\$16
CI gating that halts deployment on compliance-violating routing	Adopted as defence in depth only — see below	\$16
Explicit multi-objective reward function	Adopted in principle, rejected as specified — see below	\$16

Changed or rejected, with reasons

- **The reward weights (0.50 approval, −0.25 fraud, −0.20 cost, −0.05 latency) are rejected.** Four problems. The terms are dimensionless and therefore unvalidatable — approval and cost must be expressed in a common unit before any exchange rate between them means anything. The approval-versus-cost tradeoff is a merchant preference, not a platform constant: a subscription business and a low-margin retailer are correctly at different points, and a single global weight silently picks one. Latency as a −0.05 penalty makes the deadline negotiable, which is exactly what a constraint exists to prevent. And fraud appearing as a reward term lets routing trade fraud against approval, which is a policy decision the constraint set already owns. Replaced with a currency-denominated objective, a merchant-set optimization posture inside entitlement bounds, and latency and risk appetite retained as constraints (§16).
- **Zero shadow discrepancies over 14 days is rejected as a gate.** The new engine is meant to decide differently; a zero-divergence bar either never

- opens or is met by tuning the new engine to imitate the old one. Replaced with a two-class comparison: invariants at zero, decision divergence explained by category (§28).
- **Variance below a threshold is changed.** Variance falls when the behaviour policy was nearly deterministic, which is precisely when the logs contain no evidence about alternatives. The criterion is a confidence interval excluding harm, per objective term, per affected segment, supported by overlap and effective-sample-size diagnostics (§16).
 - **CI gating as the primary compliance control is changed.** A candidate policy should not be able to select a non-compliant acquirer at all, because the constraint set bounds the action space before optimization sees it (§06, authority level 8). Catching it in a deployment pipeline is a useful second line; treating it as the first line implies the runtime control is missing.
 - **The "dual-write" label on Phase 1 is corrected.** The mechanism described is one-way CDC mirroring, which is what §28 already specifies and is correct. Only the label was wrong, and it matters: dual-write names a pattern this design explicitly rejects.
 - **A single global write-path flip is not adopted.** Promoting the new store to primary writer for everything at once, with legacy becoming a read-only archive, discards the per-resource-class ownership model that makes rollback possible. Cutover stays per resource class, per slice (§28).

What this review did not raise

- **Outcome horizon.** A multi-objective reward is under-specified until each term declares when its outcome arrives. Approval resolves in seconds and disputes in months; a policy promoted on short-horizon evidence needs a scheduled long-horizon review and a demotion path (§16).
- **The point of no return.** Rollback is available only until the new system writes state the legacy system cannot represent. That boundary now has to be named per resource class before cutover begins, because it exists whether or not anyone writes it down (§28).
- **Objective-function ownership.** Setting the bounds on the approval-versus-cost exchange rate is a commercial decision, and it currently has no named owner. Added as open question 13 (§31).

Disposition of the enterprise architecture review (3.1)

Six recommendations were raised. Four are adopted, two are adopted with the mechanism changed, and three specific proposed mechanisms are rejected with reasons stated. The review referred to the document as v3.8; the lineage is UPOP 2.0–2.2, then specification 3.0, now 3.1.

Recommendation	Disposition	Where
Resource model imposes a cognitive cliff; provide implicit provisioning for standard checkouts	Adopted in full — and promoted to a binding constraint in §00, since a DX objection that is only answered in the SDK will be re-litigated at every API review	§00, §04, §25
Elevate cross-border and FX to a first-class domain	Adopted in full; the strongest point in the review	§03, §20
Event-sourced strangler with outbox and CDC for migration	Adopted, with the state-ownership model made explicit rather than implied	§28
Cycle detection and hard ceilings in Attempt Strategy	Adopted, extended to typed per-dimension budgets and a monotonic-progress rule	§14
Vault-time enrichment is exposed to stale data; add event-driven invalidation	Adopted in substance; mechanism changed — see below	§08, §12
Edge-cached signed plan registry with local caching	Adopted, and extended: the gap was not caching but emergency propagation, which caching does not solve	§06, §07

Rejected mechanisms, with reasons

- **A real-time cache validation call inside Context Assembly at a 1 ms budget.** A validation that must return in 1 ms is a lookup that will time out under exactly the load where correctness matters, and whose timeout behaviour is to use the cached value anyway. The latency is paid and nothing is gained. Replaced with freshness classes carried as attributes, five invalidation channels including observed contradiction, and staleness consumed as a decision input (§08).
- **A fixed seven-day verification window.** Debit network participation and issuer country do not decay at the same rate; one window is simultaneously too slow for the first and wasteful for the second. Replaced with per-attribute freshness classes (§08).
- **Deterministic fallback to single-message rails when overlay flags are absent.** Message model is a property of the merchant's capture and reversal flow. Changing it alters settlement and dispute semantics downstream, and it does not produce Reg II compliance. The correct conservative action is to suppress least-cost optimization and route to the configured default (§12).
- **Dual-write during migration, with the v3 journal as the read source of truth while legacy writes.** Dual-write has no atomicity across two stores; its failure mode is the divergence the pattern exists to prevent, appearing under load. Replaced with single-writer ownership per resource class, one-way outbox propagation, and ownership markers on every resource (§28).

What the review did not raise, and should have

- **Emergency propagation.** The review treated plan distribution as a caching and concurrency problem. Caching does not answer how a sanctions addition reaches every edge node in seconds, which is a different channel with different failure semantics. §06 adds it, fail-closed.
- **Last-known-good expiry.** Related: there was previously no configuration under which a node stopped trusting a stale artefact. There is now.

- **Cross-border substitution as a regulatory event.** The cycle-detection point is correct but understated — a substitution across an acquiring boundary changes SCA applicability and interchange treatment, so it is a new decision requiring re-evaluation, not a retry inheriting the original one (§14).

Adopted from the externally reviewed architecture document (3.0)

- **Four-tier observability framing** — the merchant / integrator / operations / engineering split is a cleaner articulation than our prior exposure matrix, extended here with the delegation mechanism it lacked.
- **Decomposed internal API surface** — the strongest section of that document; re-scoped to attempts and made shadow-only for replay.
- **Dependency versions carried inside the module execution record** — makes each record independently interpretable after the plan artefact is archived.
- **Canonical error object carrying retryability, customer action and merchant action as first-class fields** — adopted directly.
- **Industry context as an extension slot** — same instinct we had, more cleanly stated; adopted with the addition that each context declares which nodes it modifies.

Rejected from the externally reviewed architecture document (3.0)

- **A single policy engine covering entitlement, preference, capability and constraint.** Four planes with different owners and change velocities cannot share one precedence chain.
- **Single-versioned policy records.** Without bitemporality the determinism claim does not hold.
- **Policy evaluation before instrument resolution.** Makes the most common constraint class — debit, regulated debit, product-scoped — impossible to evaluate.
- **Payment Order as the top-level commercial object.** A collection wrapper cannot express obligations that outlive the collection.
- **Counterfactual outcome estimates presented as facts.** They are off-policy estimates with confidence intervals, and presenting them otherwise misleads operations at the exact moment care is needed.
- **The 40 ms latency claim.** Not achievable with mandated controls on the path; the corrected figure is under 60 ms at p99.